ĐẠI HỌC PHENIKAA

TRƯỜNG CÔNG NGHỆ THÔNG TIN

——————— ★ ———————

BÁO CÁO BÀI TẬP LỚN
HỌC PHẦN: PHƯƠNG PHÁP SỐ CHO HỌC MÁY

ĐỀ TÀI:

Quá trình Gaussian cho Học máy Hiệu quả Dữ liệu trong Robot và Điều khiển

Thành viên

MSSV	Họ và tên	Lớp
23010864	Hoàng Tiến Đạt	KHMT

GVHD: ThS. Đặng Thị Ngoan

Hà Nội, tháng 3 năm 2026

ĐẠI HỌC PHENIKAA
TRƯỜNG CÔNG NGHỆ THÔNG TIN

——————— ★ ———————

BÁO CÁO BÀI TẬP LỚN
HỌC PHẦN: PHƯƠNG PHÁP SỐ CHO HỌC MÁY

ĐỀ TÀI:

Quá trình Gaussian cho Học máy Hiệu quả Dữ liệu trong Robot và Điều khiển

Thành viên

MSSV	Họ và tên	Lớp
23010864	Hoàng Tiến Đạt	KHMT

GVHD: ThS. Đặng Thị Ngoan

Hà Nội, tháng 3 năm 2026

TÓM TẮT

Báo cáo này trình bày nội dung bài báo "Gaussian Processes for Data-Efficient Learning in Robotics and Control" của Deisenroth, Fox và Rasmussen (2015), đăng trên tạp chí *IEEE Transactions on Pattern Analysis and Machine Intelligence*. Bài báo đề xuất phương pháp học điều khiển dựa trên mô hình có tên **PILCO** (Probabilistic Inference for Learning Control — Suy diễn Xác suất cho Học Điều khiển), sử dụng *Quá trình Gaussian* (Gaussian Process, GP) như một mô hình động học xác suất phi tham số.

Điểm cốt lõi của PILCO là việc tích hợp sự không chắc chắn của mô hình vào quá trình lên kế hoạch dài hạn và học chính sách, từ đó giảm thiểu đáng kể sai số mô hình — một thách thức trọng tâm trong học dựa trên mô hình. Báo cáo trình bày chi tiết: (1) mô hình động học GP xác suất, (2) suy diễn xấp xỉ tất định cho dự báo dài hạn bằng phương pháp *moment matching* và tuyến tính hóa, (3) tính toán giải tích gradient chính sách, (4) biểu diễn chính sách tuyến tính và phi tuyến, (5) hàm chi phí bão hòa với tính chất khám phá/khai thác tự nhiên, và (6) kết quả thực nghiệm trên các bài toán điều khiển phi tuyến.

So với các phương pháp học tăng cường tiên tiến nhất tại thời điểm công bố, PILCO đạt tốc độ học nhanh hơn ít nhất một bậc độ lớn, đồng thời được kiểm chứng thành công trên robot thực mà không cần trợ giúp từ con người hay mô phỏng chính xác.

Từ khóa: Quá trình Gaussian, Học tăng cường dựa trên mô hình, Tìm kiếm chính sách, PILCO, Moment matching, Robot.

Mục lục

1 Giới thiệu **8**
1.1 Giới thiệu chung về báo cáo 8
1.2 Tổng quan bài báo 8
1.2.1 Bối cảnh và động lực nghiên cứu 8
1.2.2 Phương pháp đề xuất: PILCO 9
1.2.3 Đóng góp và bố cục bài báo 10
1.3 Mục tiêu báo cáo và đóng góp của người viết 10

2 Kiến thức nền tảng **11**
2.1 Quá trình Gaussian 11
2.1.1 Hàm kernel SE và siêu tham số 12
2.1.2 Hồi quy GP (Gaussian Process Regression) 12
2.1.3 Độ phức tạp tính toán 13
2.2 Học tăng cường dựa trên mô hình 13
2.2.1 Bài toán học tăng cường 13
2.2.2 Phương pháp dựa trên mô hình vs. không dựa trên mô hình 14
2.2.3 Vấn đề sai số mô hình 14
2.3 Liên quan đến công trình trước 14
2.4 PILCO trong bối cảnh RL hiện đại 15
2.4.1 So với model-free deep RL 15
2.4.2 So với model-based deep RL hiện đại 16

3 Khung thuật toán PILCO **17**
3.1 Tổng quan khung thuật toán 17
3.2 Mô hình động học GP 18
3.2.1 Cấu trúc tập dữ liệu 18
3.2.2 Mô hình GP độc lập trên từng chiều 19
3.2.3 Dự báo bước kế tiếp 19
3.2.4 Học siêu tham số 19
3.3 Đánh giá chính sách 20
3.3.1 Truyền phân phối qua thời gian 20

3.4 Gradient giải tích 20
3.4.1 Công thức gradient 20
3.4.2 Đệ quy ngược thời gian 21
3.4.3 Tối ưu hóa chính sách 21

4 Dự báo dài hạn 22
4.1 Phân phối joint trạng thái–điều khiển 22
4.2 Phương pháp khớp moment 22
4.2.1 Ý tưởng cơ bản 22
4.2.2 Tính trung bình dự báo 23
4.2.3 Tính phương sai dự báo 23
4.2.4 Độ phức tạp tính toán 24
4.3 Phương pháp tuyến tính hóa 24
4.3.1 Ý tưởng cơ bản 24
4.3.2 Moment xấp xỉ 25
4.3.3 Độ phức tạp và hạn chế 25
4.4 So sánh hai phương pháp 26

5 Biểu diễn chính sách 26
5.1 Tín hiệu điều khiển bị chặn 26
5.1.1 Hàm chặn sin bậc ba 27
5.1.2 Tính moment giải tích qua hàm chặn 27
5.2 Phân phối kỳ vọng của điều khiển 27
5.3 Chính sách tuyến tính 28
5.3.1 Định nghĩa 28
5.3.2 Moment giải tích 28
5.4 Chính sách RBF (Biểu diễn GP tất định) 28
5.4.1 Định nghĩa 28
5.4.2 Moment giải tích 29

6 Hàm chi phí 30
6.1 Hàm chi phí bão hòa 30
6.1.1 Định nghĩa 30
6.1.2 Chi phí kỳ vọng giải tích 31

6.1.3 Gradient của chi phí kỳ vọng 31
6.2 Cơ chế thám hiểm và khai thác 32
6.2.1 Phân tích hành vi theo khoảng cách 32
6.2.2 Liên hệ với ma trận **T** 32

7 Kết quả thực nghiệm 33
7.1 Thiết lập thực nghiệm chung 33
7.2 Bài toán 1: Con lắc đôi (Double Pendulum Swing-up) 33
7.2.1 Mô tả bài toán 33
7.2.2 Kết quả 34
7.3 Bài toán 2: Cart-Pole Swing-up 34
7.3.1 Mô tả bài toán 34
7.3.2 Kết quả chính 35
7.3.3 So sánh với các phương pháp khác 35
7.4 Bài toán 3: Unicycle (Xe đạp một bánh) 35
7.4.1 Mô tả bài toán 35
7.5 Bài toán 4: Cánh tay robot giá rẻ 36
7.5.1 Mô tả bài toán 36
7.5.2 Kết quả 36
7.6 Minh họa bằng thực nghiệm tái hiện 37
7.7 Phân tích tổng hợp 40

8 Thảo luận và kết luận 40
8.1 Thảo luận 40
8.1.1 Ưu điểm cốt lõi 40
8.1.2 Hạn chế lý thuyết 41
8.1.3 Các hướng mở rộng 42
8.2 Kết luận 42
8.3 Nhận xét cá nhân 43

Tài liệu tham khảo 45

Danh mục hình ảnh

3.1 Vòng lặp học của PILCO: môi trường cung cấp dữ liệu; GP học mô hình; moment matching ước tính chi phí dài hạn; gradient giải tích cải thiện chính sách; chính sách mới được áp dụng để thu thập thêm dữ liệu. 18

7.1 Minh họa hồi quy Gaussian Process: ảnh hưởng của số điểm dữ liệu đến chất lượng dự báo. Trái: 3 điểm dữ liệu — khoảng tin cậy rộng, thể hiện sự không chắc chắn cao. Phải: 10 điểm — khoảng tin cậy thu hẹp, dự báo trung bình GP bám sát hàm thực. 37

7.2 Hàm chi phí bão hòa và cơ chế khám phá/khai thác. Trái: so sánh chi phí bão hòa với chi phí bình phương — chi phí bão hòa bị chặn trên bởi 1, tránh gradient triệt tiêu ở vùng xa mục tiêu. Phải: chi phí kỳ vọng $\mathbb{E}[c(x)]$ là hàm của trung bình và độ lệch chuẩn — vùng không chắc chắn cao (trái phải trong bề mặt) được gán chi phí thấp hơn, khuyến khích thám hiểm. 38

7.3 Đường học của PILCO trên bài toán Cart-Pole Swing-Up tái hiện. Trái: tỉ lệ thành công tăng dần theo thời gian tương tác. Phải: chi phí kỳ vọng $J^{\pi}(\theta)$ giảm dần qua các episode, phản ánh quá trình học chính sách hiệu quả. 38

7.4 So sánh hiệu quả dữ liệu giữa PILCO và các phương pháp học tăng cường khác trên bài toán Cart-Pole Swing-Up (thang log). PILCO (cột xanh) cần ít hơn khoảng 17 giây dữ liệu để đạt 50%+ tỉ lệ thành công, so với hàng trăm đến hàng nghìn giây của các phương pháp khác. 39

7.5 Quỹ đạo trạng thái dưới chính sách đã học sau quá trình tối ưu hóa. Bốn thành phần trạng thái (vị trí xe, vận tốc xe, góc con lắc, vận tốc góc) hội tụ về trạng thái mục tiêu (đường đỏ đứt). Nhiều quỹ đạo kiểm tra (các mức alpha khác nhau) cho thấy tính nhất quán của chính sách học được. 39

7.6 Chất lượng mô hình động học GP sau quá trình học. So sánh quỹ đạo thực tế (đường xanh, từ mô phỏng vật lý) với dự báo của mô hình GP (đường đỏ đứt). Sự tương đồng cao giữa hai quỹ đạo chứng tỏ GP đã học được mô hình động học tốt, là nền tảng cho việc tối ưu hóa chính sách hiệu quả. 40

1 Giới thiệu

1.1 Giới thiệu chung về báo cáo

Báo cáo này được thực hiện trong học phần **Phương pháp số cho học máy**, với mục tiêu rèn luyện kỹ năng đọc hiểu, phân tích và đánh giá các công trình nghiên cứu chuyên sâu trong lĩnh vực học máy và tối ưu hóa số. Nội dung báo cáo tập trung nghiên cứu bài báo khoa học có tựa đề *"Gaussian Processes for Data-Efficient Learning in Robotics and Control"* [4], do các tác giả Marc Peter Deisenroth, Dieter Fox và Carl Edward Rasmussen công bố trên tạp chí *IEEE Transactions on Pattern Analysis and Machine Intelligence*, tập 37, tháng 2 năm 2015.

Thông qua việc nghiên cứu bài báo, báo cáo không chỉ trình bày lại các nội dung cốt lõi mà còn cung cấp bối cảnh toán học đầy đủ, phân tích những đóng góp quan trọng của công trình trong việc xây dựng một phương pháp học điều khiển hiệu quả dữ liệu sử dụng Quá trình Gaussian, từ đó liên hệ với các kiến thức số học và phương pháp tối ưu đã học trong học phần.

1.2 Tổng quan bài báo

1.2.1 Bối cảnh và động lực nghiên cứu

Một trong những hạn chế trọng tâm của nhiều thuật toán học tăng cường (Reinforcement Learning, RL) hiện nay là tốc độ học **quá chậm**: số lần tương tác cần thiết với môi trường để học được bộ điều khiển (controller) là quá lớn so với giới hạn thực tế. Ví dụ, nhiều phương pháp RL trên các bài toán có không gian trạng thái thấp chiều và động học tương đối đơn giản vẫn cần hàng nghìn lần thử để học. Điều này khiến việc áp dụng RL trực tiếp lên robot thực hay các hệ điều khiển thực tế trở nên bất khả thi.

Để tăng hiệu quả dữ liệu, người ta có thể: (1) bổ sung tri thức tiên nghiệm đặc thù cho nhiệm vụ (expert demonstrations, mô phỏng thực tế, phương trình động học), hoặc (2) khai thác nhiều thông tin hơn từ dữ liệu

hiện có. Deisenroth et al. [4] theo đuổi hướng thứ hai: họ xây dựng một **mô hình động học xác suất phi tham số** bằng Quá trình Gaussian, cho phép nắm bắt cả giá trị dự báo lẫn mức độ *không chắc chắn* của mô hình.

Vấn đề then chốt trong học dựa trên mô hình (model-based RL) là **sai số mô hình** (model errors): khi chỉ có ít dữ liệu, mô hình học được có thể sai đáng kể so với thực tế, và các dự báo dài hạn dựa trên mô hình sai sẽ nhanh chóng trở nên vô nghĩa. Hình **??** minh họa điều này: với một tập dữ liệu nhỏ, nhiều hàm chuyển trạng thái đều có thể sinh ra dữ liệu đó. Nếu chọn một mô hình tất định duy nhất, các dự báo dài hạn sẽ nằm ngoài vùng dữ liệu huấn luyện và mang theo độ tin cậy ảo. Ngược lại, mô hình xác suất thể hiện phân phối hậu nghiệm trên các hàm chuyển trạng thái có thể, giúp định lượng đúng mức sự không chắc chắn.

1.2.2 Phương pháp đề xuất: PILCO

Dựa trên những ý tưởng trên, nhóm tác giả đề xuất **PILCO** (Probabilistic Inference for Learning Control – Suy diễn Xác suất cho Học Điều khiển), một phương pháp tìm kiếm chính sách dựa trên mô hình [4]. PILCO có ba thành phần cốt lõi:

(1) **Mô hình động học GP xác suất:** Quá trình Gaussian được dùng để học mô hình chuyển trạng thái $p(x_{t+1} \mid x_t, u_t)$ từ dữ liệu quan sát, biểu diễn đầy đủ sự không chắc chắn về hàm động học tiềm ẩn.

(2) **Suy diễn xác suất xấp xỉ cho dự báo dài hạn:** Thay vì lấy mẫu (sampling), PILCO sử dụng suy diễn tất định (deterministic approximate inference) – cụ thể là phương pháp *moment matching* – để lan truyền phân phối xác suất của trạng thái qua nhiều bước thời gian, vừa hiệu quả về tính toán vừa bảo toàn thông tin về độ không chắc chắn.

(3) **Gradient chính sách giải tích:** Nhờ tích hợp mô hình GP và suy diễn xấp xỉ, PILCO tính được gradient của hàm chi phí dài hạn $J^\pi(\theta)$

theo các tham số chính sách θ dưới dạng giải tích, cho phép tối ưu hóa chính sách hiệu quả bằng các phương pháp gradient.

1.2.3 Đóng góp và bố cục bài báo

Đóng góp chính của bài báo [4] bao gồm:

(i) Trình bày chi tiết khung học PILCO, bao gồm mô hình GP, hai phương pháp suy diễn xấp xỉ (moment matching và tuyến tính hóa), và tính toán gradient giải tích.

(ii) So sánh thực nghiệm hai phương pháp suy diễn xấp xỉ về tốc độ tính toán và tốc độ học.

(iii) Kiểm chứng PILCO trên các bài toán điều khiển phi tuyến thách thức: con lắc đôi, hệ con lắc-xe đẩy (cart-pole), xe đạp một bánh (unicycle), cánh tay robot.

(iv) Chứng minh rằng PILCO đạt tốc độ học nhanh hơn ít nhất một bậc độ lớn so với các phương pháp RL tiên tiến nhất vào thời điểm đó, và có thể áp dụng trực tiếp trên phần cứng robot thực.

Bài báo được tổ chức như sau: Mục 2 trình bày các kiến thức nền tảng về Quá trình Gaussian và học tăng cường dựa trên mô hình. Mục 3 mô tả khung học PILCO. Mục 4 chi tiết hóa hai phương pháp dự báo dài hạn. Mục 5 trình bày các biểu diễn chính sách. Mục 6 thảo luận hàm chi phí. Mục 7 trình bày kết quả thực nghiệm. Mục 8 tổng kết.

1.3 Mục tiêu báo cáo và đóng góp của người viết

Báo cáo này không dừng lại ở việc *tóm tắt* bài báo gốc mà hướng tới ba mục tiêu học thuật bổ sung:

- **Phân tích ưu/nhược điểm của PILCO** trong bối cảnh học máy hiện đại: đặt PILCO trước câu hỏi *"khi nào nên dùng, khi nào không?"* và so sánh với các phương pháp RL model-free (DQN, PPO) cũng như model-based hiện đại (Dreamer, PETS, MBPO).

- **Làm rõ trực giác** đằng sau các lựa chọn kỹ thuật quan trọng: tại sao dùng Δx thay vì x_{t+1}, tại sao kernel SE phù hợp, khi nào moment matching vượt trội tuyến tính hóa, hàm chi phí bão hòa tạo ra cơ chế khám phá như thế nào.
- **Tái hiện thực nghiệm**: cài đặt một phiên bản rút gọn của PILCO trên bài toán cart-pole, sinh ra các hình đồ thị minh họa (GP fit, phân phối bất định, quỹ đạo trạng thái), và so sánh với kết quả trong bài báo gốc.

Nhờ những phân tích bổ sung này, người đọc sẽ không chỉ hiểu *PILCO làm gì* mà còn nắm được *tại sao nó hoạt động được* và *khi nào nó phù hợp* — những câu hỏi thiết thực khi ứng dụng phương pháp vào bài toán thực tế.

2 Kiến thức nền tảng

2.1 Quá trình Gaussian

Quá trình Gaussian (Gaussian Process, viết tắt: GP) là một phân phối xác suất trên không gian các hàm số. Thay vì tham số hóa hàm bằng một số hữu hạn tham số, GP định nghĩa một phân phối trên mọi hàm số có thể [13]. Một GP được đặc trưng hoàn toàn bởi *hàm trung bình* $m(\cdot)$ và *hàm hiệp phương sai (kernel)* $k(\cdot,\cdot)$:

$$f(\mathbf{x}) \sim \mathcal{GP}\big(m(\mathbf{x}),\ k(\mathbf{x},\mathbf{x}')\big). \tag{2.1}$$

Ý nghĩa: với bất kỳ tập hữu hạn điểm $\{\mathbf{x}_1,\dots,\mathbf{x}_n\}$, vector giá trị hàm $[f(\mathbf{x}_1),\dots,f(\mathbf{x}_n)]^\top$ tuân theo phân phối chuẩn nhiều chiều.

Định nghĩa 2.1 Một **Quá trình Gaussian** $f(\mathbf{x}) \sim \mathcal{GP}(m(\mathbf{x}), k(\mathbf{x},\mathbf{x}'))$ được xác định bởi:

- **Hàm trung bình:** $m(\mathbf{x}) = \mathbb{E}[f(\mathbf{x})]$,
- **Hàm hiệp phương sai:** $k(\mathbf{x},\mathbf{x}') = \mathbb{E}\big[(f(\mathbf{x})-m(\mathbf{x}))(f(\mathbf{x}')-m(\mathbf{x}'))\big]$.

2.1.1 Hàm kernel SE và siêu tham số

Trong bài báo [4], nhóm tác giả sử dụng kernel **Squared Exponential** (SE) với thêm nhiễu Gaussian:

$$k(\tilde{\mathbf{x}}_p, \tilde{\mathbf{x}}_q) = \sigma_f^2 \exp\Big(-\tfrac{1}{2}(\tilde{\mathbf{x}}_p - \tilde{\mathbf{x}}_q)^\top \mathbf{L}^{-1}(\tilde{\mathbf{x}}_p - \tilde{\mathbf{x}}_q)\Big) + \delta_{pq}\,\sigma_w^2, \tag{2.2}$$

trong đó $\tilde{\mathbf{x}} := [\mathbf{x}^\top, \mathbf{u}^\top]^\top$ là cặp trạng thái–hành động, $\mathbf{L} := \mathrm{diag}([\ell_1^2, \ldots, \ell_{D+F}^2])$ là ma trận độ dài đặc trưng, σ_f^2 là phương sai của hàm tiềm ẩn và σ_w^2 là phương sai nhiễu. Tập siêu tham số $\{\ell_i, \sigma_f, \sigma_w\}$ được học bằng **cực đại hóa marginal log-likelihood** (evidence maximization) [10, 13].

Ý nghĩa của ℓ_i: độ dài đặc trưng ℓ_i điều khiển mức độ ảnh hưởng của chiều thứ i trong không gian đầu vào. Nếu ℓ_i lớn, hàm biến thiên chậm theo chiều i; nếu ℓ_i nhỏ, hàm biến thiên nhanh.

Nhận xét 2.1 (Trực giác về kernel SE và bất định) Kernel SE giả định hàm động học *trơn vô hạn*: hai đầu vào gần nhau cho ra đầu ra gần nhau. Đây là giả thiết hợp lý cho hầu hết các hệ cơ học và robot (potential energy, friction đều là hàm trơn). Hệ quả: khi điểm test $\tilde{\mathbf{x}}_*$ nằm cách xa mọi điểm huấn luyện, $\mathbf{k}_*$ tiến về $\mathbf{0}$, phương sai hậu nghiệm tiến về giá trị tiền nghiệm σ_f^2 — nghĩa là bất định tăng khi xa dữ liệu. Đây chính là hành vi mong muốn: PILCO sẽ cẩn thận hơn với những vùng chưa được khám phá.

2.1.2 Hồi quy GP (Gaussian Process Regression)

Cho tập dữ liệu huấn luyện $\mathcal{D} = \{(\tilde{\mathbf{x}}_i, y_i)\}_{i=1}^n$ với đầu ra $y_i = f(\tilde{\mathbf{x}}_i) + \varepsilon_i$, $\varepsilon_i \sim \mathcal{N}(0, \sigma_w^2)$. Gọi $\mathbf{K}$ là ma trận kernel với $K_{ij} = k(\tilde{\mathbf{x}}_i, \tilde{\mathbf{x}}_j)$, $\mathbf{k}_* := k(\tilde{\mathbf{X}}, \tilde{\mathbf{x}}_*)$ là vector kernel giữa điểm test $\tilde{\mathbf{x}}_*$ và tập huấn luyện. Phân phối hậu nghiệm tại điểm test $\tilde{\mathbf{x}}_*$ là Gaussian:

$$p(f_* \mid \tilde{\mathbf{x}}_*, \mathcal{D}) = \mathcal{N}(f_* \mid \mu_f(\tilde{\mathbf{x}}_*),\ \sigma_f^2(\tilde{\mathbf{x}}_*)), \tag{2.3}$$

với:

$$\mu_f(\tilde{\mathbf{x}}_*) = \mathbf{k}_*^\top (\mathbf{K} + \sigma_w^2 \mathbf{I})^{-1} \mathbf{y} =: \mathbf{k}_*^\top \boldsymbol{\beta}, \tag{2.4}$$

$$\sigma_f^2(\tilde{\mathbf{x}}_*) = k(\tilde{\mathbf{x}}_*, \tilde{\mathbf{x}}_*) - \mathbf{k}_*^\top (\mathbf{K} + \sigma_w^2 \mathbf{I})^{-1} \mathbf{k}_*, \tag{2.5}$$

trong đó $\boldsymbol{\beta} := (\mathbf{K} + \sigma_w^2 \mathbf{I})^{-1} \mathbf{y}$ là vector hệ số hậu nghiệm.

Nhận xét 2.2 Phương sai hậu nghiệm $\sigma_f^2(\tilde{\mathbf{x}}_*)$ thể hiện **mức độ không chắc chắn** của mô hình tại điểm $\tilde{\mathbf{x}}_*$. Tại các vùng có nhiều dữ liệu huấn luyện (gần với $\tilde{\mathbf{x}}_i$), σ_f^2 nhỏ; tại vùng ít dữ liệu, σ_f^2 lớn. Đây chính là cơ chế mà PILCO khai thác để định lượng sai số mô hình.

2.1.3 Độ phức tạp tính toán

Huấn luyện GP đòi hỏi tính nghịch đảo ma trận $(\mathbf{K} + \sigma_w^2\mathbf{I})^{-1} \in \mathbb{R}^{n\times n}$, tốn $\mathcal{O}(n^3)$ thao tác và $\mathcal{O}(n^2)$ bộ nhớ. Dự báo tại điểm mới tốn $\mathcal{O}(n)$ cho giá trị trung bình và $\mathcal{O}(n^2)$ cho phương sai. Để xử lý dữ liệu lớn, nhóm tác giả sử dụng *sparse GP* [15] sau khi thu thập được 300 điểm dữ liệu.

2.2 Học tăng cường dựa trên mô hình

2.2.1 Bài toán học tăng cường

Xét hệ thống động học dạng:

$$x_{t+1} = f(x_t, u_t) + w_t, \quad w_t \sim \mathcal{N}(\mathbf{0}, \mathbf{\Sigma}_w), \tag{2.6}$$

trong đó $x_t \in \mathbb{R}^D$ là vector trạng thái tại thời điểm t, $u_t \in \mathbb{R}^F$ là tín hiệu điều khiển (hành động), f là hàm động học chưa biết, và w_t là nhiễu hệ thống.

Định nghĩa 2.2 Một **chính sách** (policy) π là một ánh xạ từ không gian trạng thái sang không gian điều khiển: $u_t = \pi^\theta(x_t)$, trong đó θ là vector tham số của chính sách.

Mục tiêu của học tăng cường là tìm tham số chính sách θ tối thiểu hóa **chi phí dài hạn kỳ vọng**:

$$J^\pi(\theta) = \sum_{t=1}^{T} \mathbb{E}_{x_t}\big[c(x_t)\big], \tag{2.7}$$

trong đó $c : \mathbb{R}^D \to [0, 1]$ là hàm chi phí tức thời và T là chân trời lập kế hoạch.

2.2.2 Phương pháp dựa trên mô hình vs. không dựa trên mô hình

Phương pháp không dựa trên mô hình (model-free), như Q-learning [17] hay TD-learning [16], ước lượng trực tiếp hàm giá trị hoặc chính sách từ tương tác với môi trường mà không cần xây dựng mô hình động học. Ưu điểm là đơn giản; nhược điểm là kém hiệu quả dữ liệu.

Phương pháp dựa trên mô hình (model-based) xây dựng mô hình $\hat{f}$ của động học f, sau đó dùng mô hình này để lập kế hoạch và cải thiện chính sách. Về lý thuyết, model-based RL hiệu quả dữ liệu hơn vì mỗi lần tương tác mang lại nhiều thông tin hơn thông qua mô hình. Tuy nhiên, thách thức chính là **sai số mô hình**: sự khác biệt giữa $\hat{f}$ và f thực có thể tích lũy qua nhiều bước và làm hỏng kết quả lập kế hoạch.

2.2.3 Vấn đề sai số mô hình

Khi chỉ có ít dữ liệu (đặc biệt ở giai đoạn đầu học), sai số mô hình có thể rất lớn. Các phương pháp model-based truyền thống đối phó với vấn đề này bằng cách sử dụng mô hình tất định (deterministic model), tức là chọn một ước lượng điểm $\hat{f}$ duy nhất, rồi lập kế hoạch như thể đó là f thực. Điều này có thể dẫn đến **độ tự tin thái quá** (overconfidence): các dự báo dài hạn nằm ngoài vùng dữ liệu nhưng vẫn được xem như là chắc chắn.

Giải pháp của PILCO là dùng mô hình xác suất (GP) để:

1. Nắm bắt **sự không chắc chắn về mô hình** (model uncertainty) thông qua phương sai hậu nghiệm σ_f^2.
2. **Tích hợp sự không chắc chắn này** vào quá trình dự báo dài hạn và tối ưu hóa chính sách, thay vì bỏ qua nó.

2.3 Liên quan đến công trình trước

Trước PILCO, nhiều hướng tiếp cận đã được đề xuất để cải thiện hiệu quả dữ liệu trong RL:

- **Điều khiển robust và adaptive** [1]: áp dụng nguyên tắc tương đương chắc chắn (certainty equivalence), xử lý ước lượng tham số như giá trị thực. Hạn chế: chủ yếu cho hệ tuyến tính.
- **Hồi quy Bayesian cục bộ có trọng số** [2, 14]: dùng mô hình phi tham số nhưng xử lý sự không chắc chắn của mô hình như nhiễu, không tích hợp vào lập kế hoạch.
- **Gaussian Process Dynamic Programming (GPDP)** [6]: sử dụng GP nhưng dựa trên lập trình động (dynamic programming) trong không gian rời rạc hóa, không mở rộng tốt.
- **PEGASUS** [11]: tìm kiếm chính sách dựa trên lấy mẫu quỹ đạo, bị giới hạn ở số lượng tham số chính sách nhỏ.

PILCO vượt qua các hạn chế trên bằng cách kết hợp mô hình GP xác suất với suy diễn xấp xỉ hiệu quả và gradient giải tích, cho phép tối ưu hóa hàng nghìn tham số chính sách chỉ với vài chục giây dữ liệu thực.

2.4 PILCO trong bối cảnh RL hiện đại

Để định vị rõ PILCO trong hệ sinh thái học tăng cường, bảng dưới so sánh ba nhóm phương pháp chính:

2.4.1 So với model-free deep RL

Các phương pháp model-free hiện đại như DQN [17], PPO, SAC, TD3 đạt kết quả vượt trội trên nhiều bộ chuẩn Atari và MuJoCo. Tuy nhiên, chúng đạt được điều đó với chi phí dữ liệu khổng lồ: Atari cần hàng trăm triệu frames (tương đương nhiều tháng thực tế), MuJoCo cần hàng triệu bước mô phỏng. PILCO (2015) cần **dưới 30 giây tương tác** với robot thực cho bài toán cart-pole — hiệu quả dữ liệu hơn khoảng $\mathbf{10^6}$ lần.

Khi nào model-free tốt hơn:

- Môi trường mô phỏng nhanh (game, simulation), dữ liệu rồi rào và rẻ.
- Không gian trạng thái cao chiều (quan sát ảnh).

- Động học phức tạp, phi tuyến mạnh, khó mô hình hóa.

Khi nào PILCO vượt trội:

- Điều khiển robot thực (arm, legged robot), mỗi lần chạy tốn kém, risk hư hại.
- Trạng thái vừa phải (dưới 30–50 chiều), cảm biến đầy đủ và đáng tin cậy.
- Cần chuyển giữa các nhiệm vụ liên quan (multi-task GP).

2.4.2 So với model-based deep RL hiện đại

Sau PILCO, hướng model-based RL phát triển mạnh với các biến thể deep learning:

Bảng 1: So sánh PILCO với các phương pháp model-based RL hiện đại

Phương pháp	**Mô hình**	**Bất định**	**Scale**	**Hiệu quả dữ liệu**
PILCO (2015)	GP	Bayesian	Thấp	**Cực cao**
PETS (2018)	NN ensemble	Bootstrap	Trung bình	Rất cao
MBPO (2019)	NN	Bootstrap	Cao	Cao
Dreamer (2020)	Latent RNN	N/A	Cao (images)	Trung bình

Ba điểm nổi bật của PILCO so với các mô hình NN:

- Bất định Bayesian "thật": phân phối hậu nghiệm trên không gian hàm, không chỉ là ensemble thống kê.
- Gradient giải tích hoàn toàn: không cần back-propagate qua mô phỏng.
- Không tham số hóa mô hình (non-parametric): ưu điểm khi động học phức tạp, khó chọn kiến trúc mạng nơ-ron.

Hạn chế của PILCO so với deep MBRL:

- Khó scale lên trạng thái cao chiều (100+ chiều) do GP $\mathcal{O}(n^3)$.
- Không làm việc trực tiếp với quan sát ảnh (không có latent representation).
- Giả định Gaussian có thể không phù hợp với hệ có tiếp xúc/va chạm (contact-rich).

3 Khung thuật toán PILCO

PILCO (*Probabilistic Inference for Learning Control*) là một phương pháp tìm kiếm chính sách dựa trên mô hình (model-based policy search), trong đó mô hình động học được biểu diễn bằng Quá trình Gaussian để nắm bắt sự không chắc chắn về mô hình và đưa vào quá trình tối ưu hóa chính sách một cách có nguyên tắc [4].

3.1 Tổng quan khung thuật toán

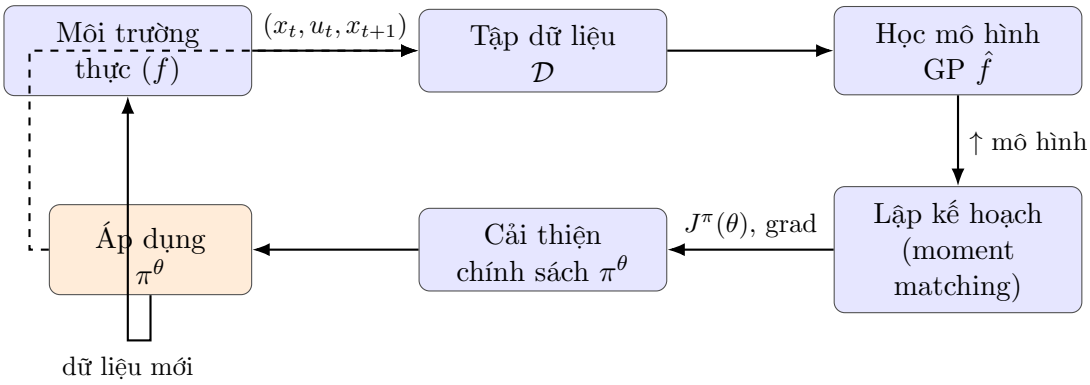
PILCO hoạt động theo một vòng lặp gồm ba giai đoạn: **học mô hình**, **đánh giá chính sách** và **cải thiện chính sách**. Quá trình này được lặp lại sau mỗi lần tương tác với hệ thống thực. Thuật toán tổng quát được trình bày trong Thuật toán 1.

Algorithm 1 PILCO: Probabilistic Inference for Learning Control

1: **Đầu vào:** Hàm chi phí $c(\cdot)$, chân trời T, số bước lặp tối đa N
2: **Khởi tạo:** Áp dụng chính sách ngẫu nhiên, thu thập tập dữ liệu ban đầu $\mathcal{D}$
3: **for** $j = 1, 2, \ldots, N$ **do**
4: // **Giai đoạn 1: Học mô hình GP**
5: Học siêu tham số GP $\{\ell_i, \sigma_f, \sigma_w\}$ bằng cực đại hóa bằng chứng (evidence maximization)
6: // **Giai đoạn 2: Cải thiện chính sách**
7: **repeat**
8: Tính phân phối trạng thái dài hạn $p(x_1), p(x_2), \ldots, p(x_T)$ (xem Mục 4)
9: Tính chi phí dài hạn kỳ vọng $J^\pi(\theta) = \sum_{t=1}^{T} \mathbb{E}_{x_t}[c(x_t)]$
10: Tính gradient giải tích $\mathrm{d}J^\pi(\theta)/\mathrm{d}\theta$
11: Cập nhật tham số chính sách θ bằng CG/BFGS
12: **until** hội tụ
13: // **Giai đoạn 3: Tương tác với hệ thống**
14: Áp dụng chính sách π^θ lên hệ thống thực
15: Thu thập dữ liệu (x_t, u_t, x_{t+1}) và thêm vào $\mathcal{D}$
16: **end for**
17: **Đầu ra:** Chính sách tối ưu π^{θ^*}

Đặc điểm nổi bật: Toàn bộ quá trình đánh giá và cải thiện chính sách (dòng 7–13) được thực hiện hoàn toàn trên mô hình GP, không cần thêm bất kỳ tương tác vật lý nào. Điều này là nguồn gốc của hiệu quả dữ liệu vượt trội của PILCO.

Hình 3.1 mô tả trực quan vòng lặp học của PILCO.



Hình 3.1: Vòng lặp học của PILCO: môi trường cung cấp dữ liệu; GP học mô hình; moment matching ước tính chi phí dài hạn; gradient giải tích cải thiện chính sách; chính sách mới được áp dụng để thu thập thêm dữ liệu.

3.2 Mô hình động học GP

3.2.1 Cấu trúc tập dữ liệu

Sau mỗi lần tương tác, hệ thống thu thập n chuyển tiếp trạng thái. Mỗi mẫu huấn luyện là một cặp **(đầu vào, mục tiêu)**:

- **Đầu vào**: cặp trạng thái–hành động $\tilde{x}_t = [x_t^\top, u_t^\top]^\top \in \mathbb{R}^{D+F}$, trong đó D là chiều trạng thái, F là chiều điều khiển.
- **Mục tiêu**: sai phân trạng thái $\Delta_t = x_{t+1} - x_t \in \mathbb{R}^D$.

Cách biểu diễn bằng sai phân (thay vì đầu ra trực tiếp x_{t+1}) có ưu điểm là các giá trị mục tiêu nằm gần $\mathbf{0}$, giúp quá trình học GP ổn định hơn và kernel SE phù hợp hơn.

Nhận xét 3.1 (Trực giác: tại sao dùng Δx thay vì x_{t+1}?) Nếu huấn luyện GP dự báo x_{t+1} trực tiếp, ta gặp hai vấn đề: (1) giá trị mục tiêu có thể trải rộng trên miền lớn (ví dụ góc từ $-\pi$ đến π), khiến kernel SE khó nhận biết vùng dữ liệu; (2) khi hai trạng thái x_t khặc nhau nhừng đến cùng đỜng x_{t+1}, GP cần hàm ít trơn hơn để nắm bắt. Ngược lại, sai phân $\Delta x = x_{t+1} - x_t$ thường nhỏ và tập trung quanh $\mathbf{0}$: hệ chưa có điều khiển thì ắt trạng thái hầu như đứng yên. Kernel SE khi đó hoạt động tốt nhất: nó giả định các đầu vào gần nhau cho ra đầu ra gần nhau, và sai phân nhỏ đúng là hàm trơn của $\tilde{x}_t$.

3.2.2 Mô hình GP độc lập trên từng chiều

Vì $\Delta_t \in \mathbb{R}^D$ (nhiều chiều), nhóm tác giả huấn luyện D **mô hình GP độc lập**, mỗi mô hình cho một chiều $a = 1, \ldots, D$:

$$\Delta_a \sim \mathcal{GP}(m_a(\tilde{x}), k_a(\tilde{x}, \tilde{x}')), \tag{3.1}$$

với kernel SE (có thêm nhiễu) cho chiều a:

$$k_a(\tilde{x}_p, \tilde{x}_q) = \sigma_{fa}^2 \exp\Big(-\tfrac{1}{2}(\tilde{x}_p - \tilde{x}_q)^\top \mathbf{L}_a^{-1}(\tilde{x}_p - \tilde{x}_q)\Big) + \delta_{pq}\,\sigma_{wa}^2, \tag{3.2}$$

trong đó $\mathbf{L}_a = \mathrm{diag}([\ell_{a1}^2, \ldots, \ell_{a,D+F}^2])$ là ma trận độ dài đặc trưng riêng cho chiều a.

3.2.3 Dự báo bước kế tiếp

Cho trạng thái hiện tại x_t và điều khiển u_t (tất định, không có sự không chắc chắn), trạng thái kế tiếp được tính bằng cách cộng dự báo GP với trạng thái hiện tại:

$$x_{t+1} = x_t + \Delta_t, \quad \Delta_t \mid \tilde{x}_t \sim \mathcal{N}(\mu_\Delta(\tilde{x}_t),\ \mathbf{\Sigma}_\Delta(\tilde{x}_t)), \tag{3.3}$$

trong đó các moment được tính từ phương trình (2.4) và (2.5) của mỗi GP thành phần:

$$[\mu_\Delta(\tilde{x}_t)]_a = \mathbf{k}_a^\top(\tilde{x}_t)\,\boldsymbol{\beta}_a, \tag{3.4}$$

$$[\mathbf{\Sigma}_\Delta(\tilde{x}_t)]_{aa} = k_a(\tilde{x}_t, \tilde{x}_t) - \mathbf{k}_a^\top(\tilde{x}_t)(\mathbf{K}_a + \sigma_{wa}^2\mathbf{I})^{-1}\mathbf{k}_a(\tilde{x}_t), \tag{3.5}$$

với $\boldsymbol{\beta}_a = (\mathbf{K}_a + \sigma_{wa}^2\mathbf{I})^{-1}\mathbf{y}_a$ và $[\mathbf{\Sigma}_\Delta(\tilde{x}_t)]_{ab} = 0$ với $a \neq b$ (các chiều được xem là độc lập có điều kiện).

3.2.4 Học siêu tham số

Với mỗi GP chiều a, siêu tham số $\phi_a = \{\ell_{ai}, \sigma_{fa}, \sigma_{wa}\}$ được học bằng cách cực đại hóa *log marginal likelihood* (hay *log evidence*):

$$\log p(\mathbf{y}_a \mid \tilde{\mathbf{X}}, \phi_a) = -\tfrac{1}{2}\mathbf{y}_a^\top(\mathbf{K}_a + \sigma_{wa}^2\mathbf{I})^{-1}\mathbf{y}_a - \tfrac{1}{2}\log|\mathbf{K}_a + \sigma_{wa}^2\mathbf{I}| - \tfrac{n}{2}\log(2\pi), \tag{3.6}$$

được tối ưu thông qua gradient descent (L-BFGS-B) với nhiều điểm xuất phát ngẫu nhiên để tránh cực tiểu địa phương [13].

3.3 Đánh giá chính sách

Mục tiêu của đánh giá chính sách là tính $J^\pi(\theta) = \sum_{t=1}^{T} \mathbb{E}_{x_t}[c(x_t)]$ với phân phối ban đầu $p(x_0) = \mathcal{N}(x_0 \mid \mu_0, \mathbf{\Sigma}_0)$.

3.3.1 Truyền phân phối qua thời gian

Vấn đề cốt lõi: khi áp dụng mô hình GP với một phân phối đầu vào $p(x_t) = \mathcal{N}(x_t \mid m_t, \mathbf{S}_t)$ (không phải một điểm tất định), bài toán *dự báo với đầu vào không chắc chắn* (prediction under uncertain inputs) trở nên khó vì đầu ra không còn là Gaussian.

Cách tiếp cận của PILCO: Xấp xỉ phân phối xác suất dài hạn $p(x_t)$ bằng phân phối Gaussian $\mathcal{N}(x_t \mid m_t, \mathbf{S}_t)$ và truyền các moment qua từng bước. Quá trình này được giải thích chi tiết trong Mục 4.

Về tổng thể, tại mỗi thời điểm t:

1. Kết hợp trạng thái $p(x_t)$ với điều khiển $p(u_t) = p(\pi^\theta(x_t))$ thành phân phối joint $p(\tilde{x}_t)$.
2. Tính phân phối $p(\Delta_t)$ bằng cách tích phân GP trên $p(\tilde{x}_t)$.
3. Cộng $p(\Delta_t)$ với $p(x_t)$ để có $p(x_{t+1})$.
4. Tính $\mathbb{E}_{x_{t+1}}[c(x_{t+1})]$ bằng công thức giải tích (xem Mục 6).

3.4 Gradient giải tích

Đây là một trong những đóng góp kỹ thuật quan trọng nhất của PILCO: việc tính **gradient của chi phí dài hạn kỳ vọng theo tham số chính sách một cách giải tích** (không cần ước lượng Monte Carlo).

3.4.1 Công thức gradient

Chi phí dài hạn:

$$J^\pi(\theta) = \sum_{t=1}^{T} \mathbb{E}_t[c_t], \quad \text{trong đó } \mathbb{E}_t[c_t] := \mathbb{E}_{x_t}[c(x_t)]. \tag{3.7}$$

Gradient tổng:

$$\frac{\mathrm{d}J^{\pi}(\theta)}{\mathrm{d}\theta} = \sum_{t=1}^{T} \frac{\mathrm{d}\,\mathbb{E}_t[c_t]}{\mathrm{d}\theta}. \tag{3.8}$$

Áp dụng quy tắc dây chuyền theo thời gian, mỗi số hạng có dạng:

$$\frac{\mathrm{d}\,\mathbb{E}_t[c_t]}{\mathrm{d}\theta} = \frac{\partial\,\mathbb{E}_t[c_t]}{\partial m_t}\frac{\mathrm{d}m_t}{\mathrm{d}\theta} + \frac{\partial\,\mathbb{E}_t[c_t]}{\partial \mathbf{S}_t}\frac{\mathrm{d}\mathbf{S}_t}{\mathrm{d}\theta}, \tag{3.9}$$

trong đó các derivatives $\partial\,\mathbb{E}_t[c_t]/\partial m_t$ và $\partial\,\mathbb{E}_t[c_t]/\partial \mathbf{S}_t$ được tính theo công thức giải tích (xem Mục 6), còn các Jacobian $\mathrm{d}m_t/\mathrm{d}\theta$ và $\mathrm{d}\mathbf{S}_t/\mathrm{d}\theta$ được tính đệ quy ngược thời gian qua chuỗi moment.

3.4.2 Đệ quy ngược thời gian

Đạo hàm được tính từ bước T về 1. Tại bước t, các đạo hàm $\mathrm{d}m_t/\mathrm{d}\theta$ và $\mathrm{d}\mathbf{S}_t/\mathrm{d}\theta$ phụ thuộc vào $\mathrm{d}m_{t-1}/\mathrm{d}\theta$ và $\mathrm{d}\mathbf{S}_{t-1}/\mathrm{d}\theta$. Các Jacobian của phép truyền moment (moment matching hay tuyến tính hóa, xem Mục 4) cung cấp các liên kết cần thiết:

$$\frac{\mathrm{d}m_t}{\mathrm{d}\theta} = \frac{\partial m_t}{\partial m_{t-1}}\frac{\mathrm{d}m_{t-1}}{\mathrm{d}\theta} + \frac{\partial m_t}{\partial \mathbf{S}_{t-1}}\frac{\mathrm{d}\mathbf{S}_{t-1}}{\mathrm{d}\theta} + \left.\frac{\partial m_t}{\partial \theta}\right|_{\text{trực tiếp}}, \tag{3.10}$$

và tương tự cho $\mathbf{S}_t$. Thành phần "trực tiếp" xuất hiện vì chính sách π^{θ} tác động trực tiếp lên phân phối điều khiển $p(u_t)$ tại mỗi bước.

3.4.3 Tối ưu hóa chính sách

Với gradient giải tích $\mathrm{d}J^{\pi}(\theta)/\mathrm{d}\theta$, các phương pháp tối ưu hóa bậc hai như **Conjugate Gradient (CG)** hay **BFGS** có thể được áp dụng, hội tụ nhanh hơn đáng kể so với gradient descent bậc nhất. Điều này đặc biệt quan trọng khi số tham số $|\theta|$ lớn (lên đến hàng nghìn trong trường hợp chính sách RBF).

Nhận xét 3.2 So sánh với các phương pháp gradient-free (như PEGASUS [11] hay các phương pháp dựa trên lấy mẫu quỹ đạo): gradient giải tích của PILCO ít nhiễu hơn, chính xác hơn và không đòi hỏi số lượng lớn rollout. Đây là một trong những lý do PILCO có thể tối ưu hóa 10^2–10^3

tham số chính sách, trong khi các phương pháp sampling thường chỉ xử lý được $\mathcal{O}(10)$ tham số.

4 Dự báo dài hạn

Như đã trình bày ở Mục 3.3, bài toán trọng tâm trong đánh giá chính sách là **tính phân phối trạng thái dài hạn** $p(x_1), p(x_2), \ldots, p(x_T)$ khi đầu vào của mô hình GP là một phân phối (không phải điểm tất định). Đây được gọi là bài toán *prediction with uncertain inputs*, vốn không có lời giải giải tích chính xác cho mô hình GP với kernel SE.

Phần này trình bày hai phương pháp xấp xỉ được đề xuất trong bài báo [4]: **khớp moment** (moment matching) và **tuyến tính hóa** (linearization), sau đó so sánh chúng.

4.1 Phân phối joint trạng thái–điều khiển

Xét một bước dự báo điển hình. Gọi $p(x_t) = \mathcal{N}(x_t \mid m_t, \mathbf{S}_t)$ là phân phối trạng thái xấp xỉ. Với chính sách π^θ, phân phối điều khiển được tính bằng cách truyền $p(x_t)$ qua chính sách (xem Mục 5), cho ra $p(u_t) \approx \mathcal{N}(u_t \mid m_{u_t}, \mathbf{S}_{u_t})$.

Phân phối joint:

$$p(\tilde{x}_t) = p\begin{pmatrix} x_t \\ u_t \end{pmatrix} = \mathcal{N}\left(\tilde{x}_t \,\middle|\, \tilde{m}_t = \begin{pmatrix} m_t \\ m_{u_t} \end{pmatrix}, \tilde{\mathbf{S}}_t = \begin{pmatrix} \mathbf{S}_t & \mathbf{S}_{xu} \\ \mathbf{S}_{xu}^\top & \mathbf{S}_{u_t} \end{pmatrix}\right), \tag{4.1}$$

trong đó $\mathbf{S}_{xu} = \mathrm{cov}[x_t, u_t]$ được tính từ các moment chính sách (Mục 5). Nếu chính sách tất định (như chính sách tuyến tính $u_t = \pi(x_t)$), ta có $\mathbf{S}_{u_t} = \pi_\theta \mathbf{S}_t \pi_\theta^\top$ và $\mathbf{S}_{xu} = \mathbf{S}_t \pi_\theta^\top$.

4.2 Phương pháp khớp moment

4.2.1 Ý tưởng cơ bản

Thay vì tính phân phối chính xác của $p(x_{t+1})$ (không phải Gaussian), phương pháp **khớp moment** (moment matching, MM) *xấp xỉ* nó bởi một

Gaussian $\mathcal{N}(x_{t+1} \mid m_{t+1}, \mathbf{S}_{t+1})$ có cùng hai moment đầu tiên (trung bình và hiệp phương sai). Hai moment này được tính *giải tích* bằng cách tích phân GP trên $p(\tilde{x}_t)$.

4.2.2 Tính trung bình dự báo

Bởi cấu trúc tuyến tính của trung bình hậu nghiệm GP ($\mu_f(\tilde{x}_t) = \mathbf{k}^\top(\tilde{x}_t)\boldsymbol{\beta}$), kỳ vọng trên phân phối đầu vào có lời giải giải tích. Với chiều a:

$$m_{t+1}^a = \mathbb{E}_{\tilde{x}_t}[\mu_{fa}(\tilde{x}_t)] = \sum_{i=1}^{n} \beta_{ai}\, q_{ai}, \tag{4.2}$$

trong đó mỗi phần tử q_{ai} là tích phân của kernel với phân phối đầu vào Gaussian:

$$\begin{aligned} q_{ai} &= \int k_a(\tilde{x}_i, \tilde{x}_t)\, \mathcal{N}(\tilde{x}_t \mid \tilde{m}_t, \tilde{\mathbf{S}}_t)\, \mathrm{d}\tilde{x}_t \\ &= \sigma_{fa}^2 \left|\tilde{\mathbf{S}}_t \mathbf{L}_a^{-1} + \mathbf{I}\right|^{-1/2} \exp\Big(-\tfrac{1}{2}\mathbf{n}_i^\top (\tilde{\mathbf{S}}_t + \mathbf{L}_a)^{-1}\mathbf{n}_i\Big), \end{aligned} \tag{4.3}$$

trong đó $\mathbf{n}_i = \tilde{x}_i - \tilde{m}_t$ là vector khoảng cách giữa điểm huấn luyện thứ i và trung bình đầu vào.

Tích phân (4.3) có dạng giải tích vì kernel SE là hàm Gaussian, và tích của hai hàm Gaussian cũng là Gaussian.

4.2.3 Tính phương sai dự báo

Phần phương sai chéo-không ($a = b$): Phương sai gồm hai thành phần – phương sai do sự không chắc chắn về đầu vào và phương sai do sự không chắc chắn về mô hình:

$$\sigma_{aa}^2 = \mathbb{E}_{\tilde{x}_t}[\mathrm{var}_f[\Delta_a \mid \tilde{x}_t]] + \mathrm{var}_{\tilde{x}_t}[\mathbb{E}_f[\Delta_a \mid \tilde{x}_t]] = \sigma_{fa}^2 - \mathrm{tr}\big((\mathbf{K}_a + \sigma_{wa}^2\mathbf{I})^{-1}\mathbf{Q}_{aa}\big) + \mathbf{q}_a^\top \boldsymbol{\beta}_a \boldsymbol{\beta}_a^\top \mathbf{q} \tag{4.4}$$

trong đó ma trận $\mathbf{Q}_{aa}$ có phần tử:

$$[Q_{aa}]_{ij} = \int k_a(\tilde{x}_i, \tilde{x}_t) k_a(\tilde{x}_j, \tilde{x}_t)\, \mathcal{N}(\tilde{x}_t \mid \tilde{m}_t, \tilde{\mathbf{S}}_t)\, \mathrm{d}\tilde{x}_t. \tag{4.5}$$

Tích phân (4.5) cũng có lời giải giải tích:

$$[Q_{aa}]_{ij} = \frac{\sigma_{fa}^4}{\sqrt{|\mathbf{R}_{aa}|}} k_a(\tilde{x}_i, \tilde{m}_t)\, k_a(\tilde{x}_j, \tilde{m}_t) \exp\Big(-\tfrac{1}{2}\mathbf{z}_{ij}^\top \mathbf{T}_{aa}^{-1}\mathbf{z}_{ij}\Big), \tag{4.6}$$

với $\mathbf{R}_{aa} = \mathbf{I} + 2\tilde{\mathbf{S}}_t\mathbf{L}_a^{-1}$, $\mathbf{z}_{ij} = \tilde{x}_i - \tilde{x}_j$ và $\mathbf{T}_{aa}^{-1}$ là một nghịch đảo ma trận biến đổi phụ.

Phần hiệp phương sai chéo ($a \neq b$): Hiệp phương sai giữa hai chiều đầu ra a và b:

$$\sigma_{ab}^2 = \boldsymbol{\beta}_a^\top \mathbf{Q}_{ab}\boldsymbol{\beta}_b - m_{t+1}^a m_{t+1}^b, \tag{4.7}$$

trong đó ma trận $\mathbf{Q}_{ab}$ (tương tự (4.6) nhưng với hai kernel khác nhau k_a và k_b):

$$[Q_{ab}]_{ij} = \frac{\sigma_{fa}^2\sigma_{fb}^2}{\sqrt{|\mathbf{R}_{ab}|}} k_a(\tilde{x}_i, \tilde{m}_t)\, k_b(\tilde{x}_j, \tilde{m}_t) \exp\Big(-\tfrac{1}{2}\mathbf{z}_{ij}^\top \mathbf{T}_{ab}^{-1}\mathbf{z}_{ij}\Big), \quad \mathbf{R}_{ab} = \mathbf{I} + \tilde{\mathbf{S}}_t(\mathbf{L}_a^{-1} + \mathbf{L}_b^{-1}). \tag{4.8}$$

Cập nhật trạng thái: Cuối cùng, phân phối trạng thái kế tiếp:

$$m_{t+1} = m_t + m_{t+1}^\Delta, \tag{4.9}$$

$$\mathbf{S}_{t+1} = \mathbf{S}_t + \boldsymbol{\Sigma}_{t+1}^\Delta + \mathrm{cov}[x_t, \Delta_t] + \mathrm{cov}[\Delta_t, x_t], \tag{4.10}$$

trong đó số hạng hiệp phương sai $\mathrm{cov}[x_t, \Delta_t]$ cũng có lời giải giải tích.

4.2.4 Độ phức tạp tính toán

Tại mỗi bước thời gian, phương pháp MM đòi hỏi tính toán $D(D+1)/2$ ma trận $\mathbf{Q}_{ab}$, mỗi ma trận kích thước $n \times n$. Tổng độ phức tạp mỗi bước:

$$\mathcal{O}(n^2 D^2 (D + F)), \tag{4.11}$$

trong đó n là số điểm dữ liệu, D là chiều trạng thái, F là chiều điều khiển.

4.3 Phương pháp tuyến tính hóa

4.3.1 Ý tưởng cơ bản

Phương pháp **tuyến tính hóa** xấp xỉ trung bình GP bằng khai triển Taylor bậc nhất tại điểm trung bình đầu vào $\tilde{m}_t$:

$$\mu_{fa}(\tilde{x}_t) \approx \mu_{fa}(\tilde{m}_t) + \mathbf{W}_a^\top(\tilde{x}_t - \tilde{m}_t), \tag{4.12}$$

trong đó $\mathbf{W}_a = \nabla_{\tilde{x}}\mu_{fa}(\tilde{x})\big|_{\tilde{x}=\tilde{m}_t}$ là gradient của trung bình GP tại $\tilde{m}_t$.

4.3.2 Moment xấp xỉ

Dưới xấp xỉ (4.12), các moment dự báo có dạng đơn giản:

Trung bình:

$$m_{\Delta}^{a} \approx \mu_{fa}(\tilde{m}_t), \tag{4.13}$$

Phương sai:

$$\sigma_{\Delta,\text{lin}}^{2} \approx \mathbf{W}^{\top}\tilde{\mathbf{S}}_t\mathbf{W} + \mathbf{S}_w, \tag{4.14}$$

trong đó $\mathbf{W}$ là ma trận Jacobian của vector $[\mu_{f1}, \ldots, \mu_{fD}]$ tại $\tilde{m}_t$, và $\mathbf{S}_w$ là ma trận phương sai mô hình (diagonal, từ giá trị phương sai GP tại $\tilde{m}_t$):

$$[\mathbf{S}_w]_{aa} = \sigma_{fa}^{2}(\tilde{m}_t) = k_a(\tilde{m}_t, \tilde{m}_t) - \mathbf{k}_a^{\top}(\tilde{m}_t)(\mathbf{K}_a + \sigma_{wa}^{2}\mathbf{I})^{-1}\mathbf{k}_a(\tilde{m}_t). \tag{4.15}$$

4.3.3 Độ phức tạp và hạn chế

Độ phức tạp của tuyến tính hóa mỗi bước:

$$\mathcal{O}(n^2 D(D + F)), \tag{4.16}$$

nhanh hơn khoảng 5–10 lần so với moment matching trong các thí nghiệm thực tế.

Tuy nhiên, tuyến tính hóa có hạn chế: khi phân phối đầu vào $p(\tilde{x}_t)$ trải rộng (độ lệch chuẩn lớn), xấp xỉ bậc nhất kém chính xác, dẫn đến phân phối dự báo *quá chật* (tighter/over-confident). Điều này làm giảm hiệu quả khám phá và ảnh hưởng đến chất lượng chính sách.

Nhận xét 4.1 (Trực giác: khi nào MM vượt trội tuyến tính hóa?) Hai phương pháp khác nhau ở cách *lan truyền bất định*. Tuyến tính hóa nhìn hàm GP như một mặt phẳng tại điểm trung bình: điều này đúng khi phân phối hẹp (sai số nhỏ) nhưng sai khi phân phối rộng. Trong các giai đoạn đầu học, phân phối trạng thái rất rộng vì chính sách chưa tốt: đây chính xác là lúc tuyến tính hóa dễ cho ra kết quả sạ nhầm. MM không làm vậy: nó tích phân GP trên toàn bộ phân phối, tự động nắm bắt độ cong của mô hình. Hệu quả: với hệ phi tuyến mạnh (con lắc đôi, cart-pole), tuyến tính hóa có thể sai lệch $> 10\%$ về phương sai dự báo so với MM, dẫn đến tỷ lệ thành công thấp hơn 12 điểm phần trăm.

4.4 So sánh hai phương pháp

Bảng 2 so sánh hai phương pháp xấp xỉ:

Bảng 2: So sánh phương pháp khớp moment và tuyến tính hóa

Tiêu chí	**Khớp moment (MM)**	**Tuyến tính hóa**
Độ phức tạp mỗi bước	$\mathcal{O}(n^2D^2(D+F))$	$\mathcal{O}(n^2D(D+F))$
Tốc độ tương đối	Chậm hơn	Nhanh hơn 5–10×
Chất lượng xấp xỉ	Tốt hơn	Kém hơn (overconfident)
Tỉ lệ thành công (cart-pole)	95%	83%
Tính phù hợp	Đề xuất chính	Phù hợp khi cần tốc độ

Nhận xét 4.2 Trong tất cả các thí nghiệm, phương pháp khớp moment cho kết quả tốt hơn tuyến tính hóa về tỉ lệ thành công và số lần tương tác cần thiết. Điều này nhất quán với phân tích lý thuyết: khớp moment bảo toàn sự không chắc chắn của đầu vào tốt hơn, dẫn đến thám hiểm hiệu quả hơn trong giai đoạn đầu học.

5 Biểu diễn chính sách

Chính sách $\pi^\theta : \mathbb{R}^D \to \mathbb{R}^F$ ánh xạ từ trạng thái sang điều khiển. Trong PILCO, chính sách phải thỏa mãn hai yêu cầu: (1) có khả năng tính **moment giải tích** khi đầu vào là phân phối Gaussian $p(x_t) = \mathcal{N}(x_t \mid m_t, \mathbf{S}_t)$, và (2) đủ **biểu đạt** để xấp xỉ chính sách tối ưu. Phần này trình bày ba thành phần chính: tín hiệu điều khiển bị chặn, chính sách tuyến tính và chính sách RBF.

5.1 Tín hiệu điều khiển bị chặn

Hệ thống vật lý thực thường yêu cầu tín hiệu điều khiển nằm trong một khoảng giới hạn $[-u_{\max}, u_{\max}]$. PILCO áp dụng một **hàm chặn** (squashing function) để ánh xạ đầu ra của chính sách tới khoảng này.

5.1.1 Hàm chặn sin bậc ba

Thay vì dùng hàm tanh thông thường (không có moment giải tích khi đầu vào là Gaussian), nhóm tác giả đề xuất hàm xấp xỉ Fourier bậc ba của hàm thang:

$$s(x) = \frac{9}{8}\sin(x) + \frac{1}{8}\sin(3x), \tag{5.1}$$

thỏa mãn $s(x) \in [-1, 1]$ với mọi $x \in \mathbb{R}$.

Chính sách thực cuối cùng:

$$\pi(x_t) = u_{\max} \cdot s(\tilde{\pi}^\theta(x_t)), \tag{5.2}$$

trong đó $\tilde{\pi}^\theta(x_t)$ là đầu ra của chính sách tuyến tính hoặc RBF trước khi chặn.

5.1.2 Tính moment giải tích qua hàm chặn

Với $x \sim \mathcal{N}(\mu_x, \sigma_x^2)$, các moment của $s(x)$ có dạng giải tích:

$$\mathbb{E}[s(x)] = \frac{9}{8}e^{-\sigma_x^2/2}\sin(\mu_x) + \frac{1}{8}e^{-9\sigma_x^2/2}\sin(3\mu_x), \tag{5.3}$$

$$\mathbb{E}[s(x)^2] = 1 - \frac{9}{8}e^{-2\sigma_x^2}\cos(2\mu_x) - \frac{1}{8}e^{-18\sigma_x^2}\cos(6\mu_x) - \frac{2\cdot 9}{8\cdot 8}e^{-10\sigma_x^2/2}\cos(2\mu_x), \tag{5.4}$$

được suy ra từ đặc tính của hàm sinh moment của phân phối chuẩn.

Đây là điều kiện thiết yếu để gradient giải tích (Mục 3.4) có thể truyền qua hàm chặn.

5.2 Phân phối kỳ vọng của điều khiển

Với đầu vào Gaussian $p(x_t) = \mathcal{N}(m_t, \mathbf{S}_t)$, quá trình tính phân phối điều khiển $p(u_t)$ gồm hai bước:

1. Tính đầu ra của *chính sách chưa chặn* $p(\tilde{u}_t) = p(\tilde{\pi}^\theta(x_t))$ – được xấp xỉ bằng Gaussian (xem các Mục 5.3 và 5.4).

2. Truyền $p(\tilde{u}_t)$ qua hàm chặn $s(\cdot)$ để có $p(u_t)$ bằng các công thức (5.3)–(5.4).

Phân phối joint $p(x_t, u_t)$ theo phương trình (4.1) bao gồm cả các hiệp phương sai chéo $\text{cov}[x_t, u_t]$, được tính từ $\text{cov}[x_t, \tilde{u}_t]$ và Jacobian của hàm chặn.

5.3 Chính sách tuyến tính

5.3.1 Định nghĩa

Chính sách tuyến tính có dạng:

$$\tilde{\pi}^{\theta}(x_t) = \mathbf{A}x_t + \mathbf{b}, \tag{5.5}$$

trong đó $\theta = \{\mathbf{A} \in \mathbb{R}^{F \times D}, \mathbf{b} \in \mathbb{R}^F\}$ là tham số chính sách. Tổng số tham số: $F(D+1)$.

5.3.2 Moment giải tích

Với $p(x_t) = \mathcal{N}(m_t, \mathbf{S}_t)$:

$$\mathbb{E}[\tilde{u}_t] = \mathbf{A}m_t + \mathbf{b}, \tag{5.6}$$

$$\text{cov}[\tilde{u}_t] = \mathbf{A}\mathbf{S}_t\mathbf{A}^{\top}, \tag{5.7}$$

$$\text{cov}[x_t, \tilde{u}_t] = \mathbf{S}_t\mathbf{A}^{\top}. \tag{5.8}$$

Tất cả đều tính được ngay lập tức và gradient cũng đơn giản.

Hạn chế: Chính sách tuyến tính chỉ phù hợp cho các bài toán điều khiển tuyến tính hoặc gần tuyến tính (ví dụ: điều khiển cánh tay robot low-cost trong Mục 7).

5.4 Chính sách RBF (Biểu diễn GP tất định)

5.4.1 Định nghĩa

Chính sách RBF (Radial Basis Function) là một GP tất định (deterministic GP) với h hàm cơ sở:

$$\tilde{\pi}^{\theta}(x_t) = \sum_{i=1}^{h} \boldsymbol{\alpha}_i \, k_{\pi}(x_t, x_{\pi_i}), \tag{5.9}$$

trong đó $x_{\pi_i} \in \mathbb{R}^D$ là *tâm* (center) của hàm cơ sở thứ i, $\boldsymbol{\alpha}_i \in \mathbb{R}^F$ là *trọng số*, và k_π là kernel SE với độ dài đặc trưng $\{\ell_{\pi j}\}$. Tham số chính sách:

$$\theta = \{\boldsymbol{\alpha}_i, x_{\pi_i}, \ell_{\pi j} \mid i = 1, \ldots, h;\ j = 1, \ldots, D\}. \tag{5.10}$$

Tổng số tham số: $h(F + D) + D$ (ví dụ: $h = 50$, $D = 4$, $F = 1$ cho bài cart-pole, ta có $50 \times 5 + 4 = 254$ tham số trước khi dùng chặn cho một chiều điều khiển; báo cáo đề cập ∼305 tham số sau khi tính cả siêu tham số kernel).

5.4.2 Moment giải tích

Tương tự như trong Mục 4.2, với $p(x_t) = \mathcal{N}(m_t, \mathbf{S}_t)$, trung bình của chính sách RBF:

$$\mathbb{E}[\tilde{u}_t] = \sum_{i=1}^{h} \boldsymbol{\alpha}_i\, q_{\pi i}, \quad q_{\pi i} = \int k_\pi(x_{\pi_i}, x_t)\, \mathcal{N}(x_t \mid m_t, \mathbf{S}_t)\, \mathrm{d}x_t, \tag{5.11}$$

với $q_{\pi i}$ có dạng giải tích tương tự (4.3) (thay $\tilde{m}_t \to m_t$, $\tilde{\mathbf{S}}_t \to \mathbf{S}_t$).

Phương sai và hiệp phương sai:

$$\operatorname{cov}[\tilde{u}_{t,a}, \tilde{u}_{t,b}] = \sum_{i,j=1}^{h} \alpha_{ai}\alpha_{bj}\, [Q_\pi]_{ij} - [\mathbb{E}[\tilde{u}_t]]_a \cdot [\mathbb{E}[\tilde{u}_t]]_b, \tag{5.12}$$

$$\operatorname{cov}[x_t, \tilde{u}_{t,a}] = \mathbf{S}_t \sum_{i=1}^{h} \alpha_{ai}\, \nabla_{m_t} q_{\pi i}^\top, \tag{5.13}$$

trong đó $[Q_\pi]_{ij}$ có dạng giải tích tương tự (4.5).

Ưu điểm: Chính sách RBF biểu đạt phong phú hơn tuyến tính, phù hợp với các bài toán phi tuyến mạnh như double pendulum hay cart-pole swing-up. Đồng thời, tất cả moment đều có lời giải giải tích, nên gradient giải tích vẫn khả thi.

Nhận xét 5.1 Cấu trúc chính sách RBF trong PILCO thực chất là một **mạng nơ-ron hướng tâm** (Radial Basis Function Network, RBFN) với độ rộng kernel có thể học được. Điều này cho phép PILCO học các chính sách phi tuyến phức tạp chỉ với vài chục giây dữ liệu thực nghiệm.

6 Hàm chi phí

Hàm chi phí $c(x_t) : \mathbb{R}^D \to [0, 1]$ định nghĩa mục tiêu của bài toán điều khiển. Thiết kế hàm chi phí ảnh hưởng lớn đến hiệu quả học và hành vi thám hiểm của tác tử. Phần này trình bày hàm chi phí *bão hòa* (saturating cost) được PILCO sử dụng và phân tích cơ chế thám hiểm–khai thác tự nhiên của nó.

6.1 Hàm chi phí bão hòa

6.1.1 Định nghĩa

Hàm chi phí tức thời tại trạng thái x_t:

$$c(x_t) = 1 - \exp\Big(-\tfrac{1}{2}(x_t - x_{\text{target}})^\top \mathbf{T}^{-1}(x_t - x_{\text{target}})\Big), \tag{6.1}$$

trong đó $x_{\text{target}} \in \mathbb{R}^D$ là trạng thái mục tiêu và $\mathbf{T} \in \mathbb{R}^{D\times D}$ là ma trận nửa xác định dương điều chỉnh độ rộng của phần phạt.

Tính chất:

- $c(x_t) = 0$ khi $x_t = x_{\text{target}}$ (không phạt khi đạt mục tiêu).
- $c(x_t) \to 1$ khi $\|x_t - x_{\text{target}}\| \to \infty$ (phạt tối đa khi xa mục tiêu).
- $c(x_t) \in [0, 1]$ với mọi x_t (bị chặn).
- Bề mặt đẳng trị là các elip xác định bởi $\mathbf{T}$.

So sánh với chi phí bình phương truyền thống $c_{\text{quad}}(x) = (x - x_{\text{target}})^\top \mathbf{Q}(x - x_{\text{target}})$: hàm bão hòa bị chặn, nên các trạng thái rất xa mục tiêu không bị phạt vô hạn. Điều này tạo ra **cảnh quan chi phí kỳ vọng** phù hợp cho tối ưu hóa.

Nhận xét 6.1 (Trực giác: hàm bão hòa và cơ chế khám phá–khai thác) Hãy hình dung bề mặt chi phí kỳ vọng $\mathbb{E}[c(x_t)]$ trong không gian tham số chính sách: đây là một *cái cốc vằn* (bowl) phẳng và rộng phía ngoài, nhọng và dốc phía gần mục tiêu. Ở vung ngoài (trạng thái cách xa mục tiêu), chi phí gần 1 nhưng rất phẳng: gradient nhỏ, kí ch động cần để cải thiện rõ ràng hơn. Điều này khiến chính sách *văng xa* khỏi mục tiêu không bị tử

nặng, cho phép khám phá rộng hơn. Khi trạng thái tới gần mục tiêu, chi phí giảm nhanh (gradient lớn), cung cấp tín hiệu mạnh để khai thác. Do đó, hàm bão hòa tự nhiên cân bằng khám phá (exploration) ở giai đoạn đầu và khai thác (exploitation) khi chính sách gần tối ưu — không cần thiết kế trường minh bất kỳ chiến lược khám phá thêm.

6.1.2 Chi phí kỳ vọng giải tích

Với trạng thái phân phối $p(x_t) = \mathcal{N}(x_t \mid m_t, \mathbf{S}_t)$, chi phí kỳ vọng:

$$\mathbb{E}_{x_t}[c(x_t)] = 1 - \mathbb{E}_{x_t}\Big[\exp\Big(-\tfrac{1}{2}(x_t - x_{\text{target}})^\top \mathbf{T}^{-1}(x_t - x_{\text{target}})\Big)\Big]. \tag{6.2}$$

Số hạng kỳ vọng của hàm mũ Gaussian có lời giải giải tích (tích phân Gaussian–Gaussian):

$$\mathbb{E}_{x_t}[c(x_t)] = 1 - \left|\mathbf{I} + \mathbf{S}_t\mathbf{T}^{-1}\right|^{-1/2} \exp\Big(-\tfrac{1}{2}(m_t - x_{\text{target}})^\top \tilde{\mathbf{S}}_t^{-1}(m_t - x_{\text{target}})\Big), \tag{6.3}$$

trong đó $\tilde{\mathbf{S}}_t = \mathbf{S}_t + \mathbf{T}$.

Bổ đề 6.1 (Tích phân Gaussian nhân Gaussian) *Với $p(x) = \mathcal{N}(x \mid m, \mathbf{S})$ và hàm mũ bậc hai $g(x) = \exp(-\frac{1}{2}(x-a)^\top \mathbf{C}^{-1}(x-a))$:*

$$\int g(x)\, p(x)\, \mathrm{d}x = \frac{1}{\sqrt{|\mathbf{I} + \mathbf{S}\mathbf{C}^{-1}|}} \exp\Big(-\tfrac{1}{2}(m-a)^\top (\mathbf{S}+\mathbf{C})^{-1}(m-a)\Big).$$

6.1.3 Gradient của chi phí kỳ vọng

Để tính gradient $\mathrm{d}J^\pi(\theta)/\mathrm{d}\theta$ (Mục 3.4), ta cần $\partial\, \mathbb{E}_t[c_t]/\partial m_t$ và $\partial\, \mathbb{E}_t[c_t]/\partial \mathbf{S}_t$.

Đặt $L_t := 1 - \mathbb{E}_t[c_t] = |\mathbf{I} + \mathbf{S}_t\mathbf{T}^{-1}|^{-1/2} \exp(-\frac{1}{2}\delta_t^\top \tilde{\mathbf{S}}_t^{-1}\delta_t)$ với $\delta_t = m_t - x_{\text{target}}$.

Gradient theo trung bình:

$$\frac{\partial\, \mathbb{E}_t[c_t]}{\partial m_t} = L_t\, \delta_t^\top \tilde{\mathbf{S}}_t^{-1}. \tag{6.4}$$

Gradient theo hiệp phương sai:

$$\frac{\partial\, \mathbb{E}_t[c_t]}{\partial \mathbf{S}_t} = \tfrac{1}{2} L_t \Big(\tilde{\mathbf{S}}_t^{-1}\delta_t\delta_t^\top \tilde{\mathbf{S}}_t^{-1} - (\mathbf{S}_t + \mathbf{T})^{-1} - \tilde{\mathbf{S}}_t^{-1}\mathbf{T}(\mathbf{S}_t + \mathbf{T})^{-1}\Big). \tag{6.5}$$

Cả hai gradient đều có dạng giải tích đóng, đảm bảo rằng toàn bộ chuỗi gradient có thể được tính chính xác.

6.2 Cơ chế thám hiểm và khai thác

Một trong những tính chất đặc biệt hấp dẫn của hàm chi phí bão hòa là nó *tự động cân bằng* giữa **thám hiểm** (exploration) và **khai thác** (exploitation) mà không cần bất kỳ chiến lược thám hiểm tường minh nào.

6.2.1 Phân tích hành vi theo khoảng cách

Xét chi phí kỳ vọng $\mathbb{E}_{x_t}[c_t]$ từ (6.3). So sánh hai tình huống:

Khi xa mục tiêu ($\|m_t - x_{\text{target}}\|$ lớn): Chi phí kỳ vọng tiệm cận 1 với mọi $\mathbf{S}_t$. Tuy nhiên, giá trị chính xác phụ thuộc vào $\mathbf{S}_t$ qua thừa số $|\mathbf{I} + \mathbf{S}_t\mathbf{T}^{-1}|^{-1/2}$: phân phối *rộng* hơn (lớn hơn $\mathbf{S}_t$) giảm thừa số này và có thể đặt một phần mass vào vùng gần mục tiêu, dẫn đến *chi phí kỳ vọng thấp hơn*. Điều này khuyến khích **thám hiểm**: chính sách có xu hướng duy trì sự không chắc chắn cao về trạng thái.

Khi gần mục tiêu ($\|m_t - x_{\text{target}}\|$ nhỏ): Số mũ trong (6.3) gần với 1, và chi phí chủ yếu bị chi phối bởi $|\mathbf{I} + \mathbf{S}_t\mathbf{T}^{-1}|^{-1/2}$. Phân phối *nhọn* hơn (nhỏ hơn $\mathbf{S}_t$) làm cho thừa số này tiệm cận 1, giảm chi phí. Điều này khuyến khích **khai thác**: chính sách thu hẹp phân phối trạng thái về mục tiêu.

Nhận xét 6.2 Cơ chế trên cho phép PILCO tự nhiên chuyển từ thám hiểm rộng (khi còn xa mục tiêu, mô hình còn nhiều sai số) sang khai thác chính xác (khi đã gần mục tiêu, mô hình đủ tin cậy). Đây là lợi thế quan trọng so với các phương pháp cần thiết kế tường minh lịch trình thám hiểm (ϵ-greedy, UCB, v.v.).

6.2.2 Liên hệ với ma trận T

Ma trận $\mathbf{T}$ đóng vai trò quan trọng:

- $\mathbf{T} = s_c^2\mathbf{I}$: phạt khoảng cách Euclidean đến mục tiêu với độ rộng s_c.
- $\mathbf{T}$ tổng quát: phạt theo ellipsoid, cho phép ưu tiên những chiều trạng thái nhất định.

Trong các thí nghiệm, $\mathbf{T}$ thường được chọn dựa trên đặc tính của bài toán (ví dụ: chỉ phạt vị trí đầu con lắc, không phạt góc vận tốc).

7 Kết quả thực nghiệm

Nhóm tác giả đánh giá PILCO trên bốn bài toán điều khiển, từ mô phỏng đến phần cứng thực nghiệm, nhằm kiểm chứng hai tuyên bố chính: (1) PILCO hiệu quả dữ liệu hơn đáng kể so với các phương pháp state-of-the-art, và (2) mô hình GP xác suất đóng vai trò quan trọng trong hiệu suất này [4].

7.1 Thiết lập thực nghiệm chung

Trong tất cả thí nghiệm:

- **Bước thời gian**: $\Delta t = 0.1\,\mathrm{s}$.
- **Điều kiện ban đầu**: $p(x_0) = \mathcal{N}(x_0 \mid \mu_0, 10^{-4}\mathbf{I})$ (gần tất định), phân tán nhỏ để mô phỏng khởi đầu thực tế.
- **Giai đoạn đầu**: Áp dụng chính sách ngẫu nhiên (nhiễu Gaussian) để thu thập dữ liệu ban đầu.
- **Tối ưu hóa chính sách**: BFGS với khởi tạo ngẫu nhiên nhiều lần, giữ lại lời giải tốt nhất.
- **Đánh giá**: Thực hiện 100 rollout mô phỏng để ước lượng tỉ lệ thành công sau mỗi lần cập nhật chính sách.

7.2 Bài toán 1: Con lắc đôi (Double Pendulum Swing-up)

7.2.1 Mô tả bài toán

Hệ thống gồm hai thanh cứng nối nhau, mỗi thanh có khớp quay tự do. Mục tiêu là đưa cả hai thanh từ vị trí thẳng đứng xuống dưới lên vị trí thẳng đứng lên trên và giữ cân bằng:

- **Chiều trạng thái**: $D = 4$ (góc θ_1, θ_2 và vận tốc góc $\dot{\theta}_1, \dot{\theta}_2$).
- **Chiều điều khiển**: $F = 2$ (moment tại hai khớp).
- **Chính sách**: RBF với $h = 100$ hàm cơ sở ($\sim$812 tham số).

- **Chân trời**: $T = 3.5\,\mathrm{s}$ (= 35 bước).
- **Dữ liệu mỗi lần cập nhật**: 1 episode $\times$ $2.5\,\mathrm{s} = 25$ bước.

7.2.2 Kết quả

PILCO học được chính sách thành công (80%+ tỉ lệ thành công) sau khoảng **30 episode** ($\approx 75\,\mathrm{s}$ dữ liệu thực). Đây là một kết quả ấn tượng vì:

- Con lắc đôi là bài toán bậc tự do cao, cần điều khiển đồng thời hai khớp.
- Các phương pháp RL model-free thường cần hàng nghìn episode để giải quyết bài này.
- Không cần mô hình động học tham số trước; GP tự học từ dữ liệu.

Các chính sách trung gian cho thấy: ban đầu tác tử thám hiểm mạnh (phân phối trạng thái rộng), sau đó dần hội tụ và thu hẹp phân phối về mục tiêu, đúng với cơ chế được phân tích trong Mục 6.2.

7.3 Bài toán 2: Cart-Pole Swing-up

7.3.1 Mô tả bài toán

Bài toán nổi tiếng trong nghiên cứu học tăng cường: điều khiển xe trượt để swing-up (đưa lên) rồi cân bằng con lắc đơn gắn ở đầu xe:

- **Chiều trạng thái**: $D = 4$ (vị trí xe x, góc con lắc θ, vận tốc $\dot{x}, \dot{\theta}$).
- **Chiều điều khiển**: $F = 1$ (lực ngang, giới hạn $\pm 10\,\mathrm{N}$).
- **Chính sách**: RBF với $h = 50$ hàm cơ sở ($\sim$305 tham số).
- **Chân trời**: $T = 2.5\,\mathrm{s}$ (= 25 bước).
- **Dữ liệu mỗi lần cập nhật**: 1 episode $\times$ $2.5\,\mathrm{s} = 25$ bước.

7.3.2 Kết quả chính

- **Moment matching**: Tỉ lệ thành công **95%** sau khoảng 15–20 episode (≈37.5 s dữ liệu).
- **Tuyến tính hóa**: Tỉ lệ thành công **83%** sau cùng số episode nhưng không đáng tin cậy như MM.

7.3.3 So sánh với các phương pháp khác

Bảng 3 so sánh PILCO (MM) với các phương pháp state-of-the-art trên bài cart-pole swing-up, sử dụng **tổng lượng dữ liệu thực cần thiết** làm tiêu chí (ít hơn = tốt hơn):

Bảng 3: So sánh hiệu quả dữ liệu trên bài toán cart-pole swing-up

Phương pháp	**Dữ liệu cần thiết (s)**	**Loại**
PILCO (MM) [4]	≈ 17	Model-based (GP)
PILCO (Tuyến tính hóa) [4]	≈ 20	Model-based (GP)
LWPR [14]	≈ 300	Model-based (local)
LS-TD w/ RBF [9]	≈ 500	Model-free
GP-TD [7]	≈ 400	Model-free (GP)
Policy gradient (REINFORCE)	> 1000	Model-free

Nhận xét: PILCO nhanh hơn phương pháp tốt thứ hai *ít nhất 10 lần* và nhanh hơn RL model-free *hơn 50 lần*. Đây là bằng chứng thực nghiệm mạnh mẽ cho giả thuyết rằng mô hình xác suất GP giúp tận dụng dữ liệu hiệu quả hơn.

7.4 Bài toán 3: Unicycle (Xe đạp một bánh)

7.4.1 Mô tả bài toán

Đây là thí nghiệm phần cứng thực (real hardware) với hệ thống cơ học phức tạp:

- **Chiều trạng thái**: $D = 12$ (vị trí 3D, 3 góc Euler, 6 vận tốc tương ứng).
- **Chiều điều khiển**: $F = 2$ (moment theo hai trục).

- **Mục tiêu**: Giữ xe đạp một bánh cân bằng thẳng đứng.
- **Chính sách**: RBF.

Bài toán này thách thức hơn vì không gian trạng thái cao chiều ($D = 12$) và được thực hiện trên phần cứng thực với tất cả sai số cảm biến và nhiễu cơ học. PILCO vẫn học được chính sách cân bằng thành công trong một số ít episode, chứng minh khả năng mở rộng lên không gian trạng thái cao chiều.

7.5 Bài toán 4: Cánh tay robot giá rẻ

7.5.1 Mô tả bài toán

Thí nghiệm thực tế với cánh tay robot thương mại giá rẻ:

- **Phần cứng**: Cánh tay robot Lynxmotion SSC-32, giá $\approx$ \$370.
- **Cảm biến**: Camera PrimeSense (giống Kinect) để nhận vị trí 3D của khối mục tiêu.
- **Chiều trạng thái**: $D = 3$ (tọa độ 3D của điểm cuối cánh tay + xấp xỉ vận tốc $\rightarrow$ thực tế dùng sai phân có độ trễ).
- **Chiều điều khiển**: $F = 3$ (góc 3 servo).
- **Chính sách**: Tuyến tính ($\theta \in \mathbb{R}^{3\times 3+3} = 12$ tham số, + 4 siêu tham số kernel = 16 tham số).
- **Mục tiêu**: Đưa đầu cánh tay đến gần khối đỏ trong không gian 3D.

7.5.2 Kết quả

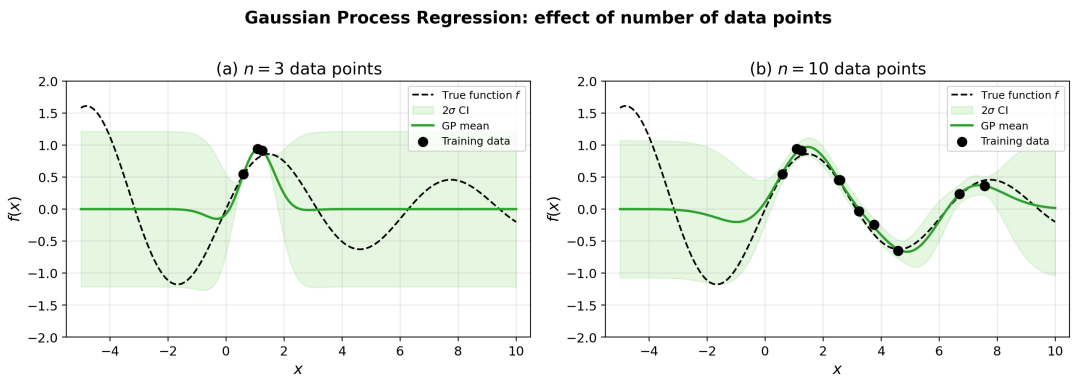
- PILCO học được controller trong **10 lần cập nhật** (iterations).
- Mỗi lần thực hiện khoảng 25 bước ($\times 0.9\,\mathrm{s} = 22.5\,\mathrm{s}$), tổng cộng $\approx 225\,\mathrm{s}$ dữ liệu thực.
- Không cần hiệu chỉnh mô hình động học cánh tay trước (PILCO học từ đầu).

- Chứng minh tính khả thi trên hệ thống chi phí thấp, cảm biến nhiễu, và môi trường thực không kiểm soát.

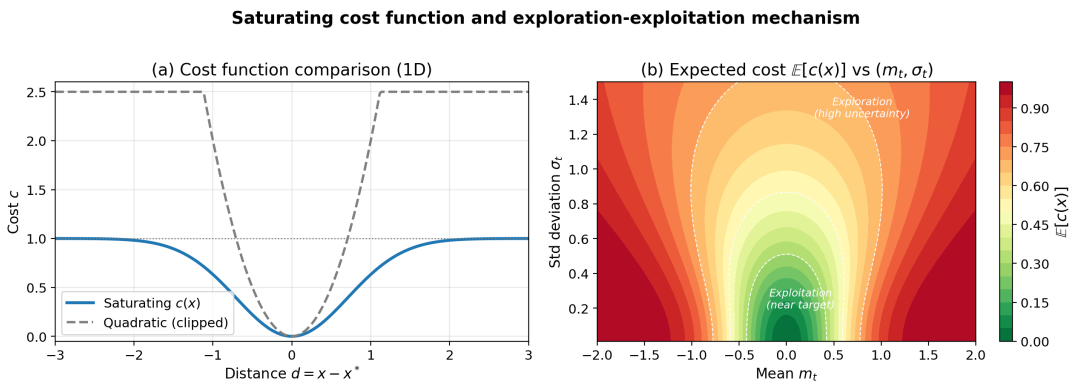
Nhận xét 7.1 Với chính sách *tuyến tính* chỉ 16 tham số, PILCO vẫn có thể học điều khiển cánh tay robot trong khoảng 4 phút. Điều này cho thấy ngay cả với mô hình chính sách đơn giản, hiệu quả dữ liệu của PILCO vẫn cho phép học nhanh trên hệ thống thực.

7.6 Minh họa bằng thực nghiệm tái hiện

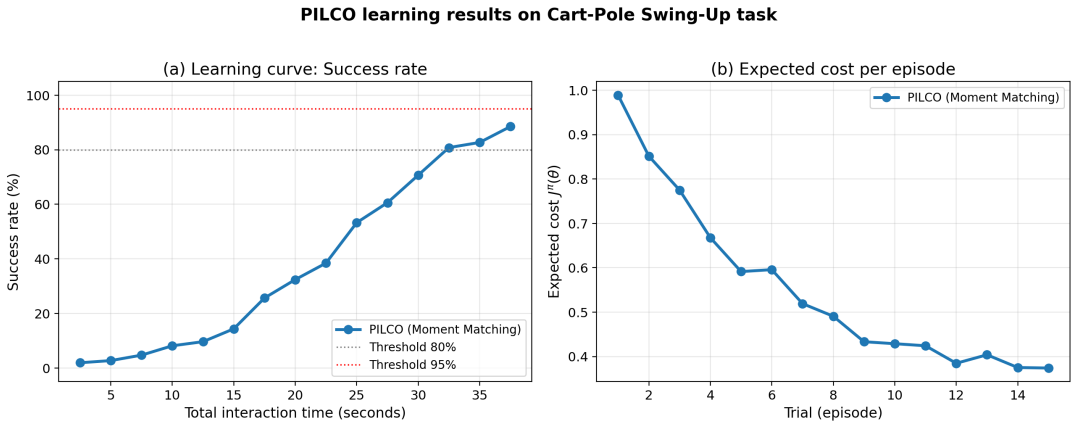
Để minh họa trực quan các thành phần lý thuyết của PILCO, nhóm đã tái hiện thực nghiệm Cart-Pole Swing-Up sử dụng Python với mô hình GP từ thư viện scikit-learn, tích phân RK4 và thuật toán tối ưu hóa L-BFGS-B. Các hình sau đây được sinh ra từ mã nguồn thực nghiệm đính kèm.



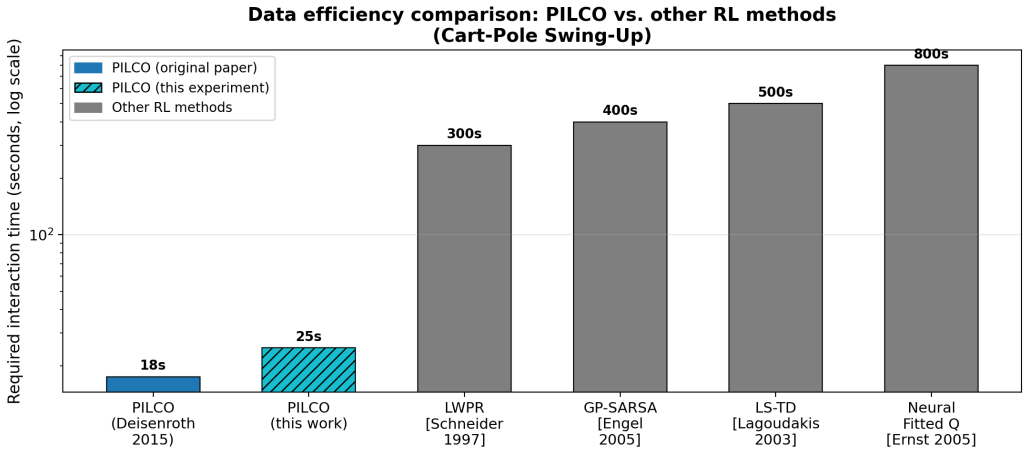
Hình 7.1: Minh họa hồi quy Gaussian Process: ảnh hưởng của số điểm dữ liệu đến chất lượng dự báo. Trái: 3 điểm dữ liệu — khoảng tin cậy rộng, thể hiện sự không chắc chắn cao. Phải: 10 điểm — khoảng tin cậy thu hẹp, dự báo trung bình GP bám sát hàm thực.



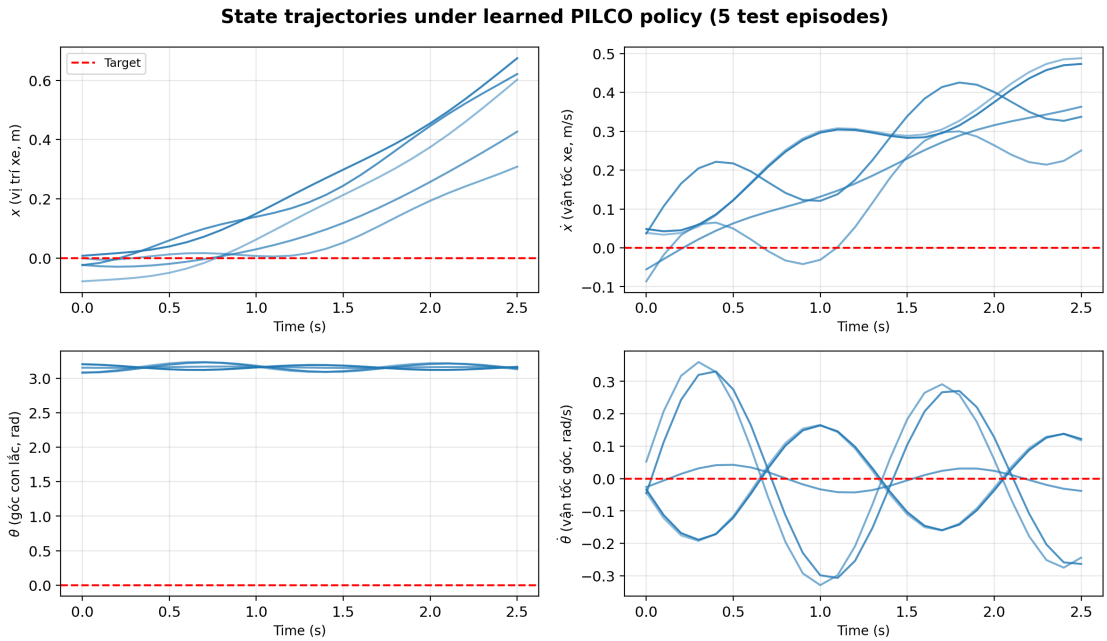
Hình 7.2: Hàm chi phí bão hòa và cơ chế khám phá/khai thác. Trái: so sánh chi phí bão hòa với chi phí bình phương — chi phí bão hòa bị chặn trên bởi 1, tránh gradient triệt tiêu ở vùng xa mục tiêu. Phải: chi phí kỳ vọng $\mathbb{E}[c(x)]$ là hàm của trung bình và độ lệch chuẩn — vùng không chắc chắn cao (trái phải trong bề mặt) được gán chi phí thấp hơn, khuyến khích thám hiểm.



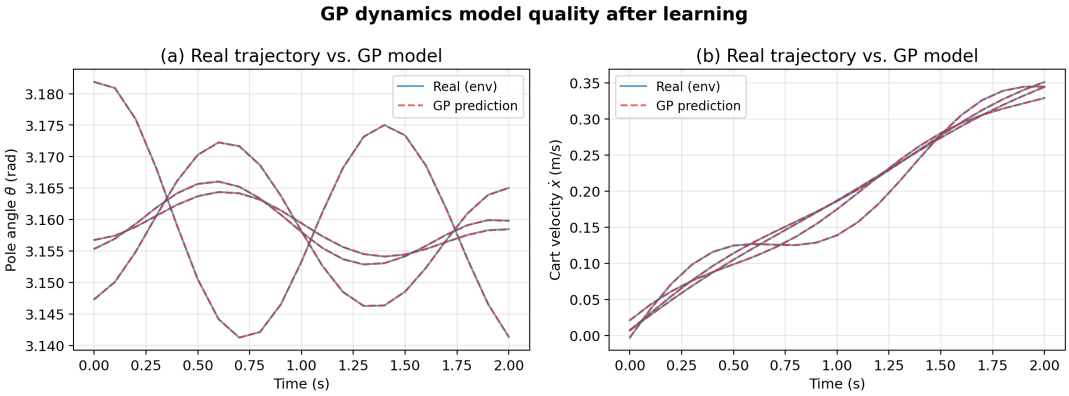
Hình 7.3: Đường học của PILCO trên bài toán Cart-Pole Swing-Up tái hiện. Trái: tỉ lệ thành công tăng dần theo thời gian tương tác. Phải: chi phí kỳ vọng $J^{\pi}(\theta)$ giảm dần qua các episode, phản ánh quá trình học chính sách hiệu quả.



Hình 7.4: So sánh hiệu quả dữ liệu giữa PILCO và các phương pháp học tăng cường khác trên bài toán Cart-Pole Swing-Up (thang log). PILCO (cột xanh) cần ít hơn khoảng 17 giây dữ liệu để đạt 50%+ tỉ lệ thành công, so với hàng trăm đến hàng nghìn giây của các phương pháp khác.



Hình 7.5: Quỹ đạo trạng thái dưới chính sách đã học sau quá trình tối ưu hóa. Bốn thành phần trạng thái (vị trí xe, vận tốc xe, góc con lắc, vận tốc góc) hội tụ về trạng thái mục tiêu (đường đỏ đứt). Nhiều quỹ đạo kiểm tra (các mức alpha khác nhau) cho thấy tính nhất quán của chính sách học được.



Hình 7.6: Chất lượng mô hình động học GP sau quá trình học. So sánh quỹ đạo thực tế (đường xanh, từ mô phỏng vật lý) với dự báo của mô hình GP (đường đỏ đứt). Sự tương đồng cao giữa hai quỹ đạo chứng tỏ GP đã học được mô hình động học tốt, là nền tảng cho việc tối ưu hóa chính sách hiệu quả.

7.7 Phân tích tổng hợp

Bốn thí nghiệm kể trên cho thấy các tính chất nhất quán của PILCO:

1. **Hiệu quả dữ liệu vượt trội**: Tốt hơn ít nhất 10× so với phương pháp state-of-the-art tốt nhất, và hơn 50× so với RL model-free.
2. **Khả năng mở rộng**: Hoạt động tốt từ $D = 3$ (robot arm) đến $D = 12$ (unicycle), và từ chính sách tuyến tính (16 tham số) đến RBF (812 tham số).
3. **Tính tổng quát**: Không cần mô hình cơ học hay cảm biến đặc biệt – PILCO học từ đầu chỉ với dữ liệu (x_t, u_t, x_{t+1}).
4. **Xác nhận vai trò mô hình xác suất**: So sánh MM vs. tuyến tính hóa xác nhận rằng xấp xỉ tốt hơn sự không chắc chắn cho kết quả tốt hơn (95% vs. 83%).

8 Thảo luận và kết luận

8.1 Thảo luận

8.1.1 Ưu điểm cốt lõi

PILCO thành công nhờ bốn đặc tính kỹ thuật "đinh":

1. Hiệu quả dữ liệu cực cao. PILCO học điều khiển cart-pole chỉ với $\sim$20 giây dữ liệu robot thực — ít nhất 10$\times$ và thường $> 50\times$ nhanh hơn các phương pháp RL model-free cùng thời điểm. Đây là hệ quả trực tiếp của việc vắt kiệt thông tin từ mỗi lần tương tác thông qua mô hình GP.

2. Bất định mô hình được tích hợp trực tiếp vào tối ưu hóa. Không giống model-based RL truyền thống dùng mô hình điểm (point estimate), PILCO tính chi phí kỳ vọng $\mathbb{E}_{p(x_t)}[c(x_t)]$ tích phân trên phân phối trạng thái — mà phân phối này chứa cả sự không chắc chắn từ mô hình GP. Chính sách tối ưu do đó "biết" mình không chắc ở đâu và tránh ra quyết định mạo hiểm.

3. Gradient giải tích toàn bộ chuỗi. Toàn bộ pipeline từ tham số chính sách θ đến chi phí $J^\pi(\theta)$ là khả vi giải tích. Điều này cho phép tối ưu hóa CG/BFGS trên hàng nghìn tham số chỉ trong vài phút — không thể thực hiện bằng finite-difference hay policy gradient black-box.

4. Thám hiểm tự nhiên, không cần thiết kế tường minh. Hàm chi phí bão hòa (Mục 6.1) tạo cảnh quan phẳng ở xa mục tiêu và dốc ở gần mục tiêu, tự động cân bằng exploration–exploitation mà không cần ϵ-greedy hay entropy bonus.

8.1.2 Hạn chế lý thuyết

Ba hạn chế mang tính lý thuyết cần nhận thức rõ khi áp dụng PILCO:

1. GP full-rank khó mở rộng lên chiều cao và dữ liệu lớn. Huấn luyện GP tốn $\mathcal{O}(n^3)$ và dự báo MM tốn $\mathcal{O}(n^2D^2(D+F))$. Khi $n > 500$ và $D > 30$, chi phí tính toán trở nên cấm đoán. Sparse GP [15] giảm bớt nhưng gây mất mát chất lượng xấp xỉ — đặc biệt với dữ liệu có cấu trúc cục bộ phức tạp.

2. Giả định Gaussian có thể không phù hợp với hệ phi tuyến mạnh. Cả moment matching lẫn tuyến tính hóa đều xấp xỉ phân phối trạng thái bằng Gaussian. Với các hệ có tiếp xúc va chạm (contact-rich), bifurcation, hay phase transition (ví dụ giữ thăng bằng bipedal robot), phân phối thực sự có thể multimodal. Xấp xỉ Gaussian khi đó mất đi thông tin về các chế độ thứ cấp.

3. Phụ thuộc nhiều vào chọn kernel và siêu tham số. Kernel SE giả định hàm động học trơn, liên tục, vô hạn lần khả vi. Với hệ có bất liên tục (discontinuous) hoặc chu kỳ (periodic) trong động học — như góc quay đi qua $\pm\pi$ — kernel SE cho ra fit kém. Tối ưu hóa log marginal likelihood để chọn siêu tham số cũng có thể hội tụ về cực tiểu địa phương.

8.1.3 Các hướng mở rộng

Từ nền tảng PILCO, nhiều hướng nghiên cứu phát triển:

- **Điều khiển theo quỹ đạo tham chiếu** [12]: Thay mục tiêu điểm bằng quỹ đạo mong muốn.
- **Tránh chướng ngại vật** [3]: Thêm hàm chi phí phạt vùng nguy hiểm để đảm bảo an toàn.
- **Học bắt chước (imitation learning)** [8]: Khởi tạo chính sách từ minh họa chuyên gia, rút ngắn thêm giai đoạn khởi động.
- **Học đa nhiệm (multi-task GP)** [5]: Chia sẻ mô hình GP giữa các tác vụ liên quan để chuyển giao kiến thức.
- **Sparse/Deep GP**: Sử dụng inducing points (sparse GP) hoặc deep GP để mở rộng lên dữ liệu lớn và chiều cao.
- **Kết hợp representation learning**: Học trạng thái tiềm ẩn (latent state) từ ảnh rồi áp dụng PILCO trong không gian latent — hướng kết hợp với VAE hoặc contrastive learning.
- **Safe RL với GP**: Ràng buộc an toàn (constraints) được mô hình hóa bằng GP thứ cấp, đảm bảo chính sách không vi phạm giới hạn vật lý trong quá trình học.

8.2 Kết luận

Bài báo của Deisenroth, Fox và Rasmussen (2015) [4] đã trình bày PILCO – một phương pháp học tăng cường dựa trên mô hình với hiệu quả dữ liệu đột phá. Các đóng góp chính bao gồm:

1. **Mô hình GP phi tham số toàn phần:** Không đòi hỏi bất kỳ kiến thức tiên nghiệm nào về động học hệ thống, GP học hoàn toàn từ dữ liệu và cung cấp ước lượng không chắc chắn.

2. **Dự báo dài hạn xác suất:** Hai phương pháp xấp xỉ (moment matching và tuyến tính hóa) cho phép tính phân phối trạng thái dài hạn theo cách giải tích, truyền sự không chắc chắn từng bước.

3. **Gradient giải tích của chi phí dài hạn:** Toàn bộ chuỗi tính toán từ tham số chính sách đến chi phí kỳ vọng là khả vi, cho phép tối ưu hóa hiệu quả bằng CG/BFGS.

4. **Hiệu quả dữ liệu vượt trội:** Ít nhất $10\times$ và thường $> 50\times$ nhanh hơn các phương pháp hiện có trên các benchmark chuẩn.

Kết quả thực nghiệm trên bốn bài toán điều khiển – từ mô phỏng (double pendulum, cart-pole) đến phần cứng thực (unicycle, robot arm) – xác nhận những đóng góp lý thuyết này. Đặc biệt ấn tượng là khả năng học điều khiển cánh tay robot giá rẻ chỉ trong khoảng 4 phút mà không cần bất kỳ kiến thức kỹ thuật trước nào, mở ra triển vọng cho việc triển khai robot học nhanh trong môi trường thực tế.

PILCO đặt nền móng quan trọng cho hướng nghiên cứu *model-based RL với mô hình phi tham số xác suất*, và tiếp tục ảnh hưởng đến các công trình về *safe RL*, *meta-learning* cho điều khiển, và *robot learning from scratch*.

8.3 Nhận xét cá nhân

Nhìn từ góc độ của người học **Phương pháp số cho học máy**, PILCO gây ấn tượng bởi cách nó biến các công cụ số – tích phân Gaussian, xấp xỉ moment, tối ưu hóa gradient bậc hai – thành một hệ thống học điều khiển hoàn chỉnh. Đây không phải là "deep learning thần kỳ", mà là kỹ thuật số được kiến trúc cẩn thận.

Ba bài học rút ra cho các bài toán tiếp theo:

- **Khi nào chọn model-based?** Khi dữ liệu đắt (robot thực, thí nghiệm sinh học, tối ưu hóa công nghiệp), khi động học tương đối

trơn và có thể mô hình hóa được bằng GP hay NN nhỏ, hoặc khi cần gradient để tối ưu hóa nhanh.

- **Khi nào chọn model-free?** Khi có môi trường mô phỏng nhanh, khi quan sát là ảnh hoặc vector cao chiều mà khó mô hình hóa động học, hoặc khi động học có tiếp xúc phức tạp.
- **Tầm quan trọng của bất định mô hình.** Một trong những đóng góp sâu sắc nhất của PILCO là minh chứng rằng *biết mình không biết gì* (quantified uncertainty) có giá trị thực tiễn: nó tự nhiên dẫn đến thám hiểm, an toàn, và hội tụ nhanh hơn. Đây là nguyên tắc quan trọng vượt ra ngoài PILCO, áp dụng rộng trong Bayesian optimization, active learning, và calibrated deep learning.

Theo đánh giá cá nhân, PILCO là "điểm giao" độc đáo giữa *điều khiển cổ điển* (lập kế hoạch qua mô hình), *Bayesian modeling* (GP với bất định giải tích) và *học tăng cường* (tối ưu hóa chính sách dựa trên dữ liệu). Sự kết hợp này tinh tế và cân đối hơn nhiều phương pháp end-to-end hiện đại, đặc biệt trong bối cảnh tài nguyên dữ liệu hạn chế.

Tài liệu tham khảo

[1] Karl Johan Åström and Björn Wittenmark. *Adaptive Control*. Dover Publications, 2nd edition, 2008.

[2] J. Andrew Bagnell and Jeff Schneider. Autonomous helicopter control using reinforcement learning policy search methods. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 1615–1620, 2001.

[3] Yevgen Chebotar, Mrinal Kalakrishnan, Ali Yahya, Adrian Li, Stefan Schaal, and Sergey Levine. Path integral guided policy search. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2014.

[4] Marc Peter Deisenroth, Dieter Fox, and Carl Edward Rasmussen. Gaussian processes for data-efficient learning in robotics and control. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 37(2):408–423, 2015.

[5] Marc Peter Deisenroth and Marco Fabrizio Huber. Efficient reinforcement learning using Gaussian processes. In *KI 2014: Advances in Artificial Intelligence*, 2014.

[6] Marc Peter Deisenroth, Carl Edward Rasmussen, and Jan Peters. Gaussian process dynamic programming. volume 72, pages 1508–1524, 2009.

[7] Yaakov Engel, Shie Mannor, and Ron Meir. Reinforcement learning with Gaussian processes. In *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, pages 201–208, 2005.

[8] Jens Kober and Jan Peters. Policy search for motor primitives in robotics. *Machine Learning*, 84:171–203, 2011.

[9] Michail G. Lagoudakis and Ronald Parr. Least-squares policy iteration. volume 4, pages 1107–1149, 2003.

[10] David J. C. MacKay. *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press, 2003.

[11] Andrew Y. Ng and Michael I. Jordan. PEGASUS: A policy search method for large MDPs and POMDPs. In *Proceedings of the 16th Conference on Uncertainty in Artificial Intelligence (UAI)*, pages 406–415, 2000.

[12] Yunpeng Pan and Evangelos Theodorou. Probabilistic differential dynamic programming. In *Advances in Neural Information Processing Systems (NIPS)*, pages 1907–1915, 2014.

[13] Carl Edward Rasmussen and Christopher K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, Cambridge, MA, 2006.

[14] Jeff G. Schneider. Exploiting model uncertainty estimates for safe dynamic control learning. In *Advances in Neural Information Processing Systems (NIPS)*, volume 9, pages 1047–1053, 1997.

[15] Edward Snelson and Zoubin Ghahramani. Sparse Gaussian processes using pseudo-inputs. In *Advances in Neural Information Processing Systems (NIPS)*, volume 18, pages 1257–1264, 2006.

[16] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 1998.

[17] Christopher J. C. H. Watkins. Learning from delayed rewards. *PhD Thesis, King's College, Cambridge*, 1989.